

**G1, ZGC, Shenandoah, etc.
With all these GCs in Java,
which one do I choose?**

Antoine DESSAIGNE

GCs in Java in 2026

G1

ZGC

Shenandoah

Serial

Parallel

???



axway 

74Software

What is a GC for?

Allocate
memory

Free
memory

What is a GC for?

Allocate
objects

Free
objects

Memory? Objects?

```
var conf = new Conference("Conference", 2026);
```

Memory? Objects?

```
var conf = new Conference("Conference", 2026);
```



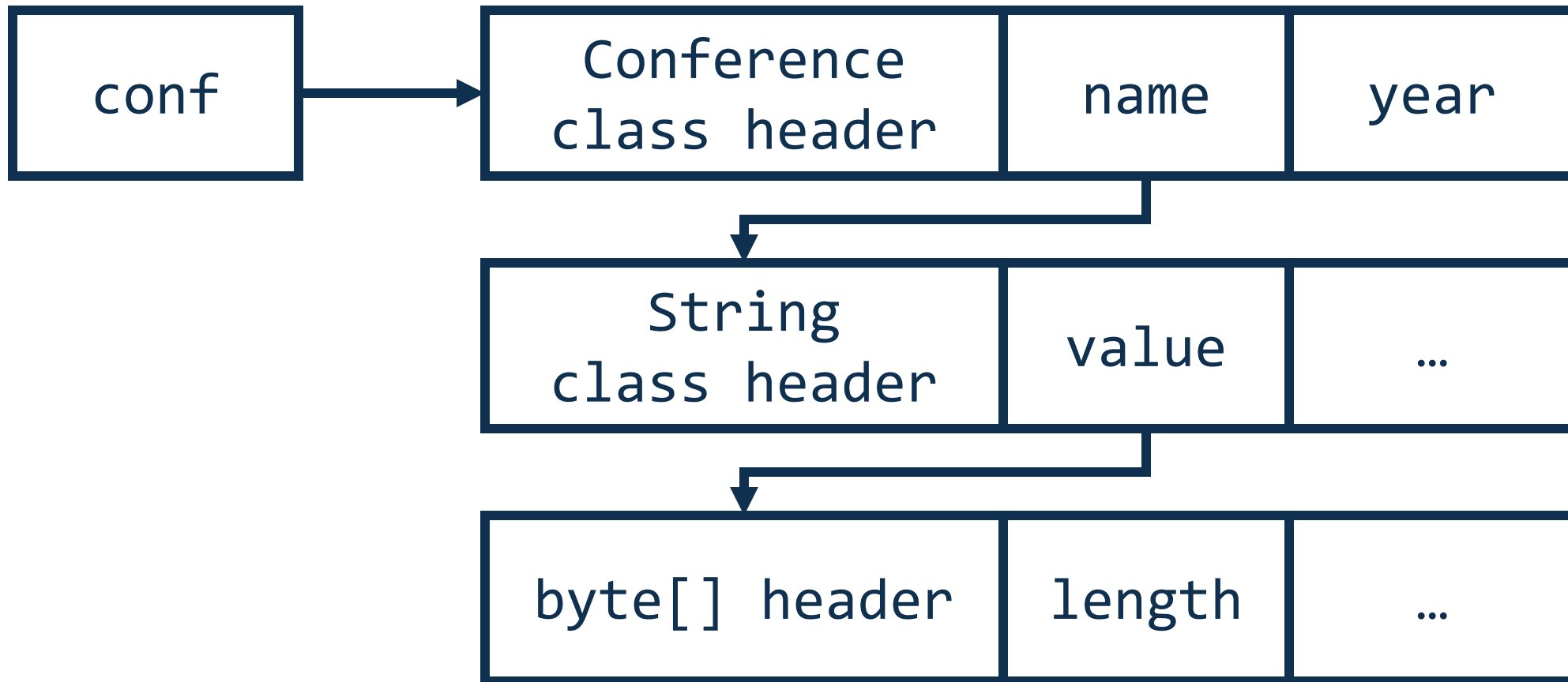
Memory? Objects?

```
var conf = new Conference("Conference", 2026);
```



Memory? Objects?

```
var conf = new Conference("Conference", 2026);
```



Memory in Java



In the beginning Java 1



CPU: 100 to 166 MHz

Memory: 8 to 16 MB

Storage: 500 to 1200 MB

Anyhow

Java 1



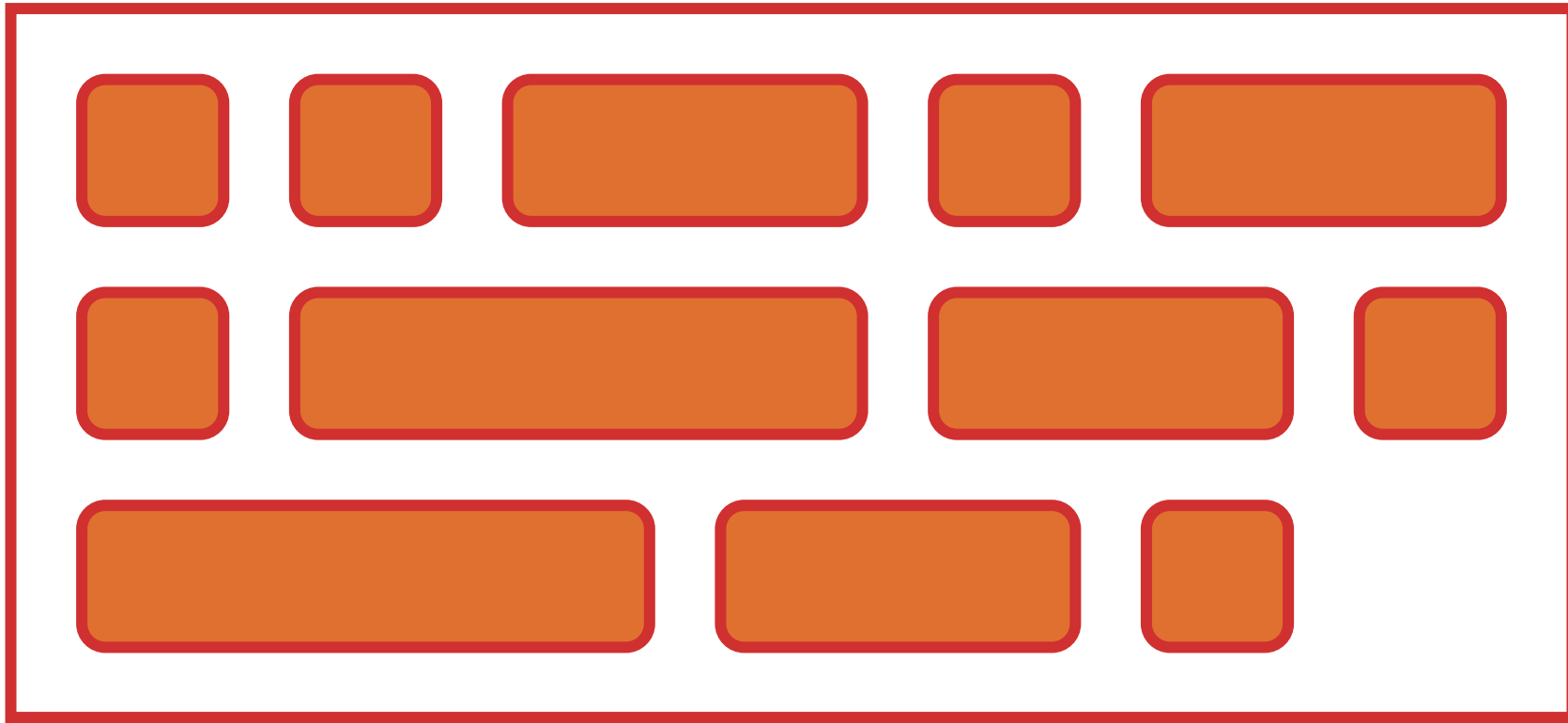
Java 1



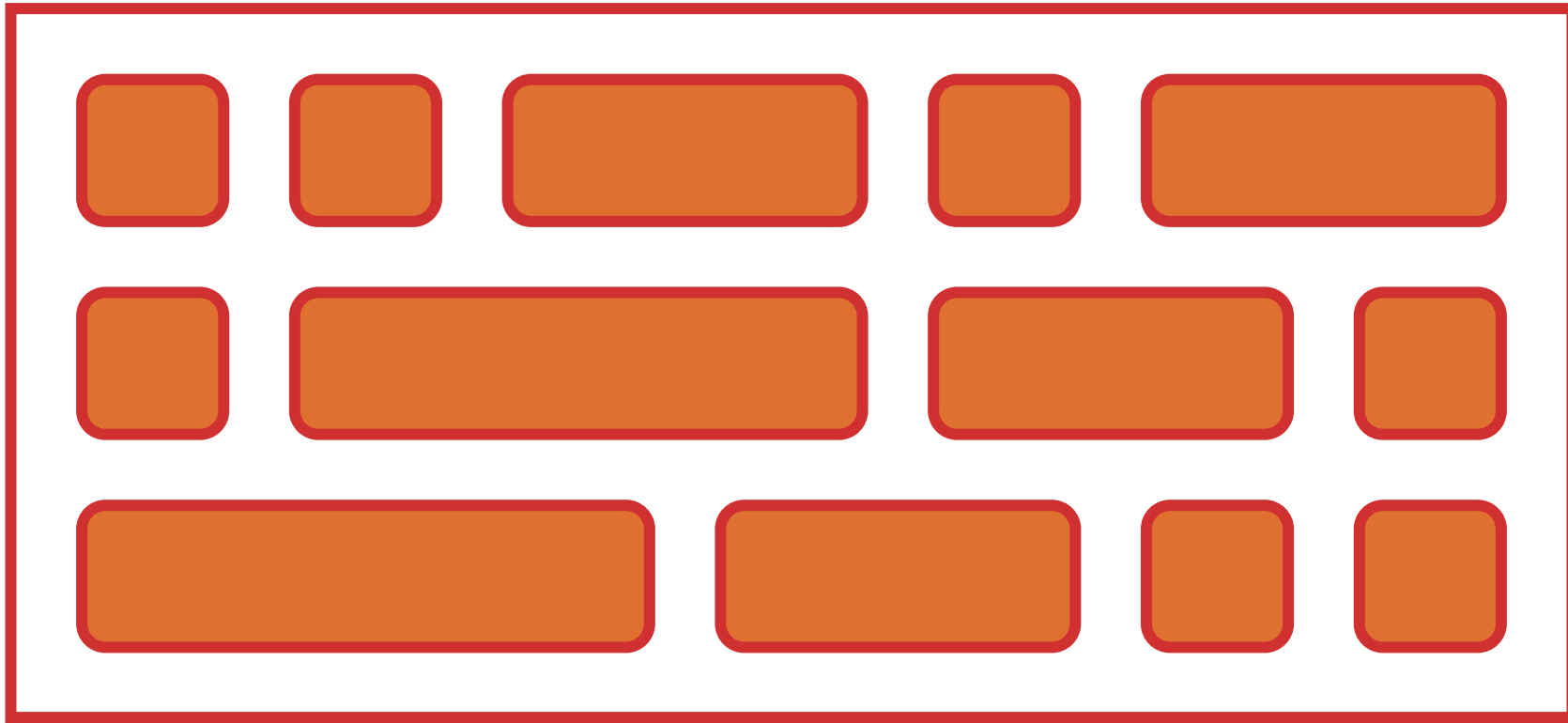
Java 1



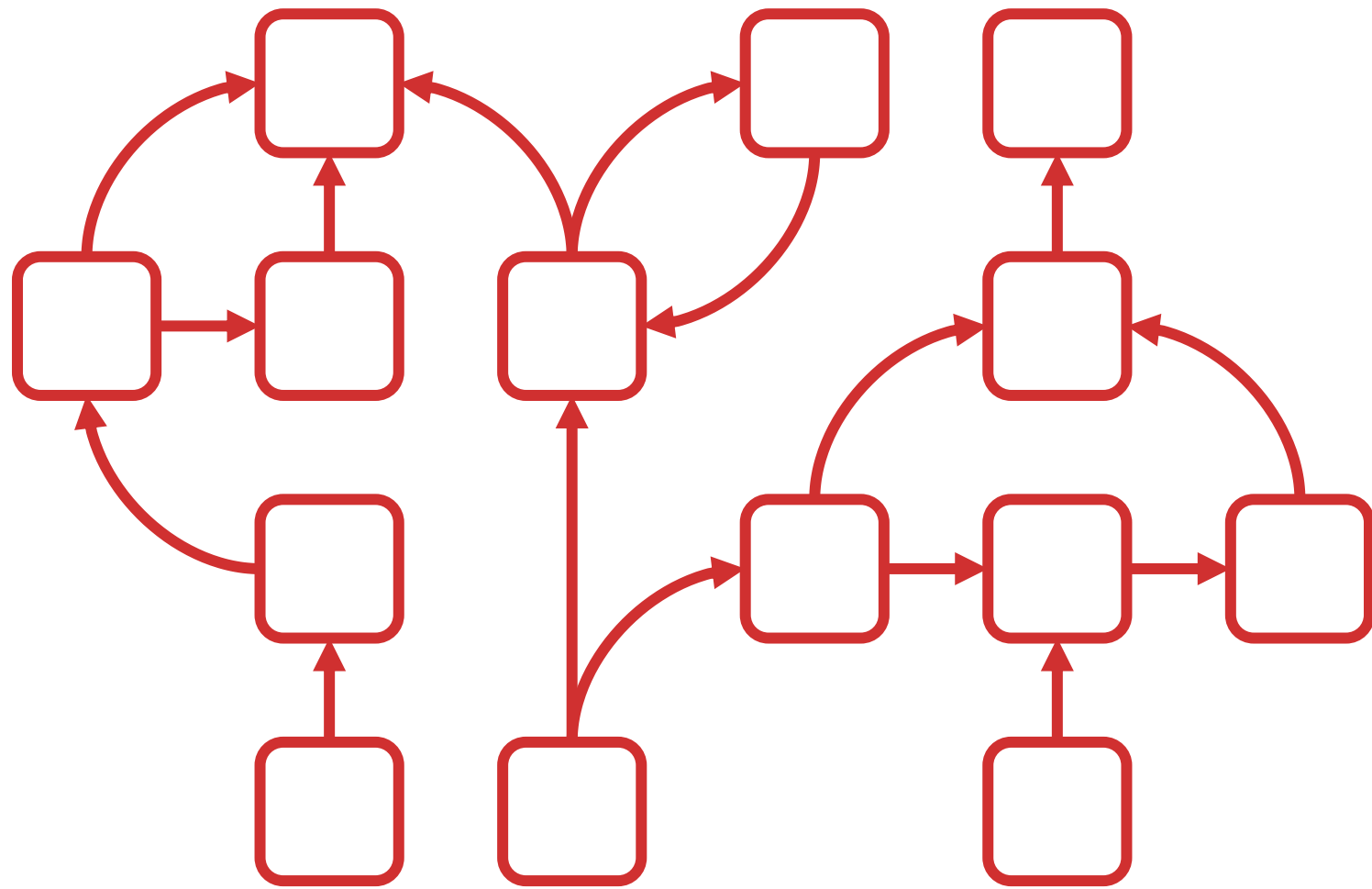
Java 1

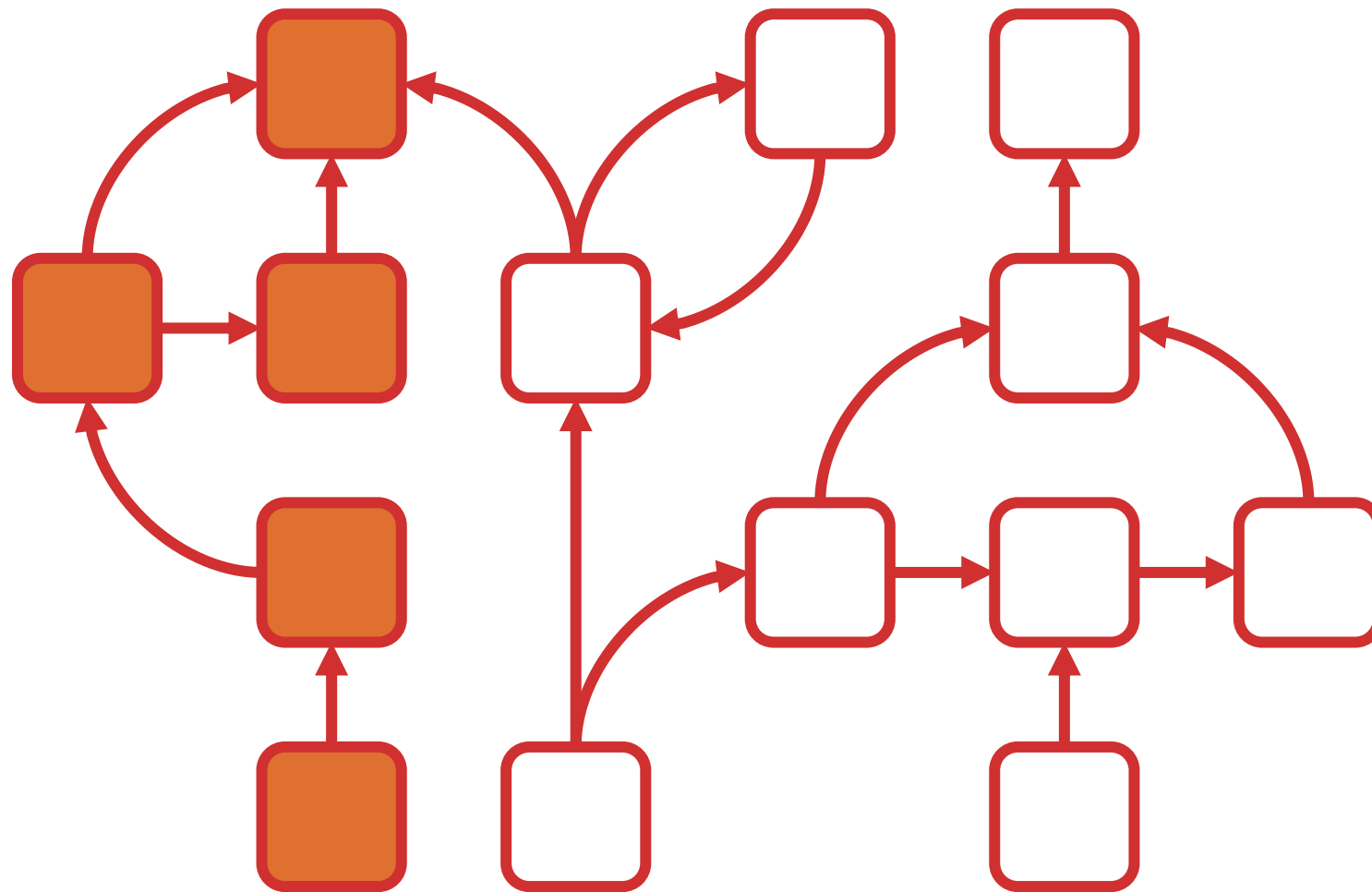


Java 1

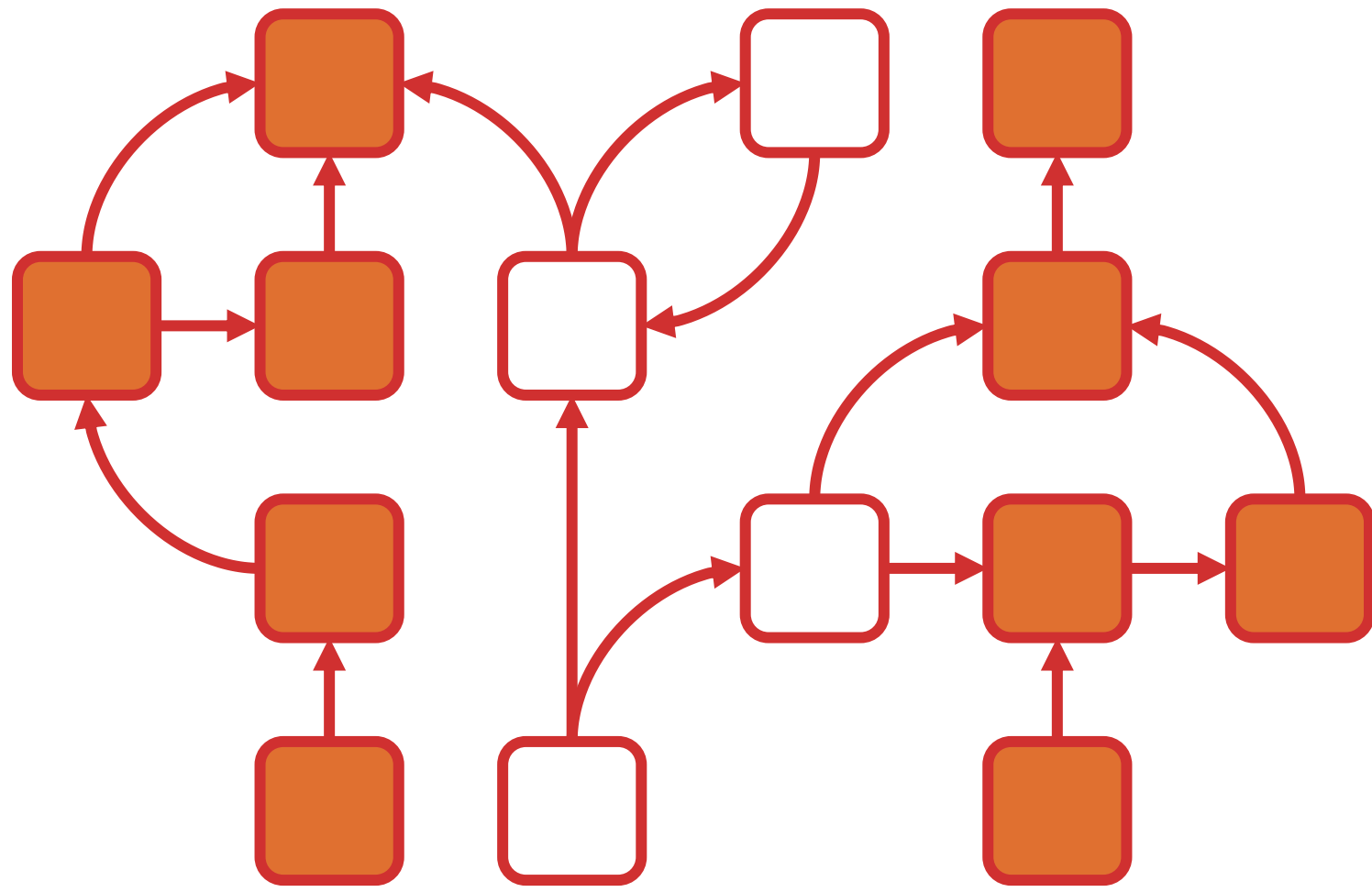


**How do we know whether
an object is still in use?**



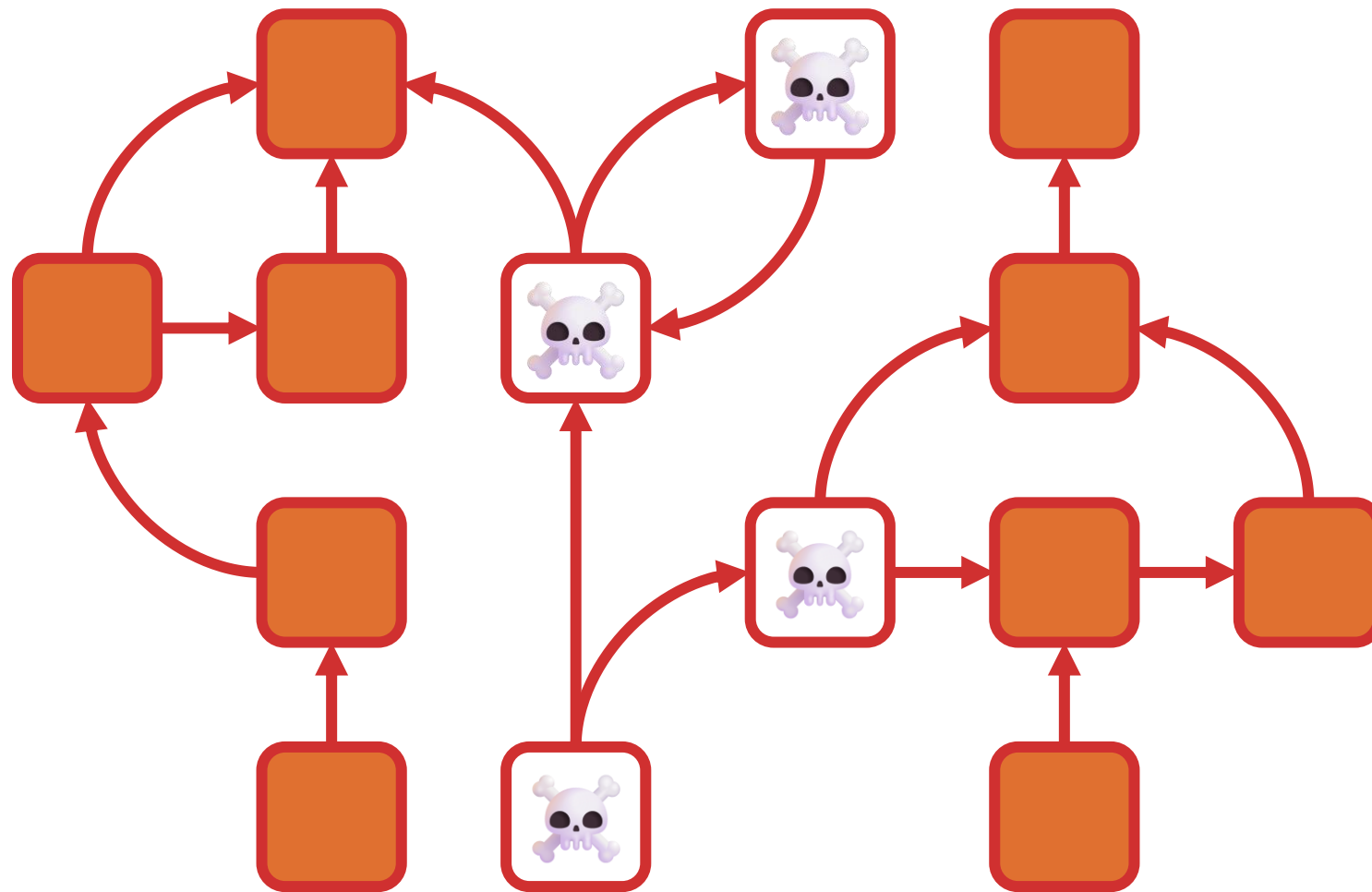


Local variables



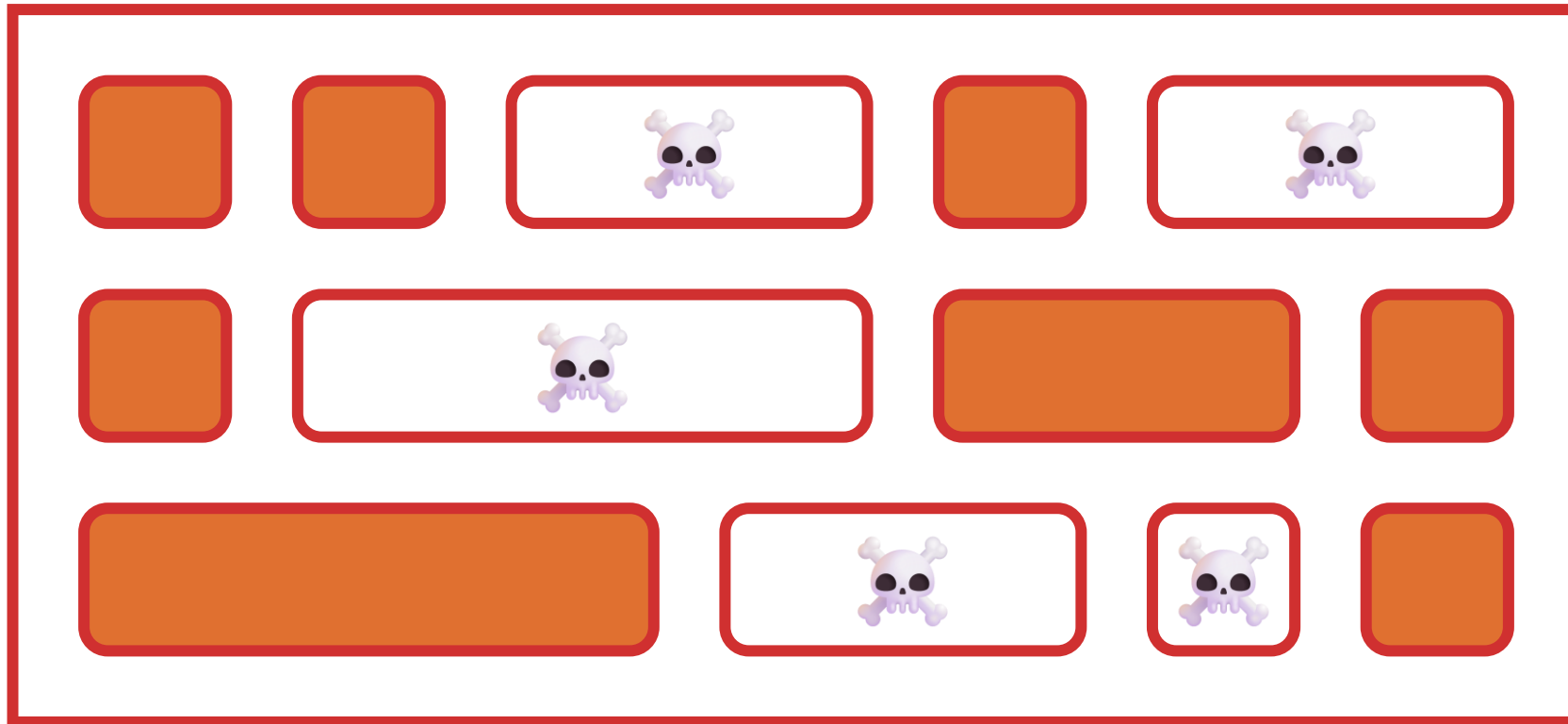
Local variables

Static fields

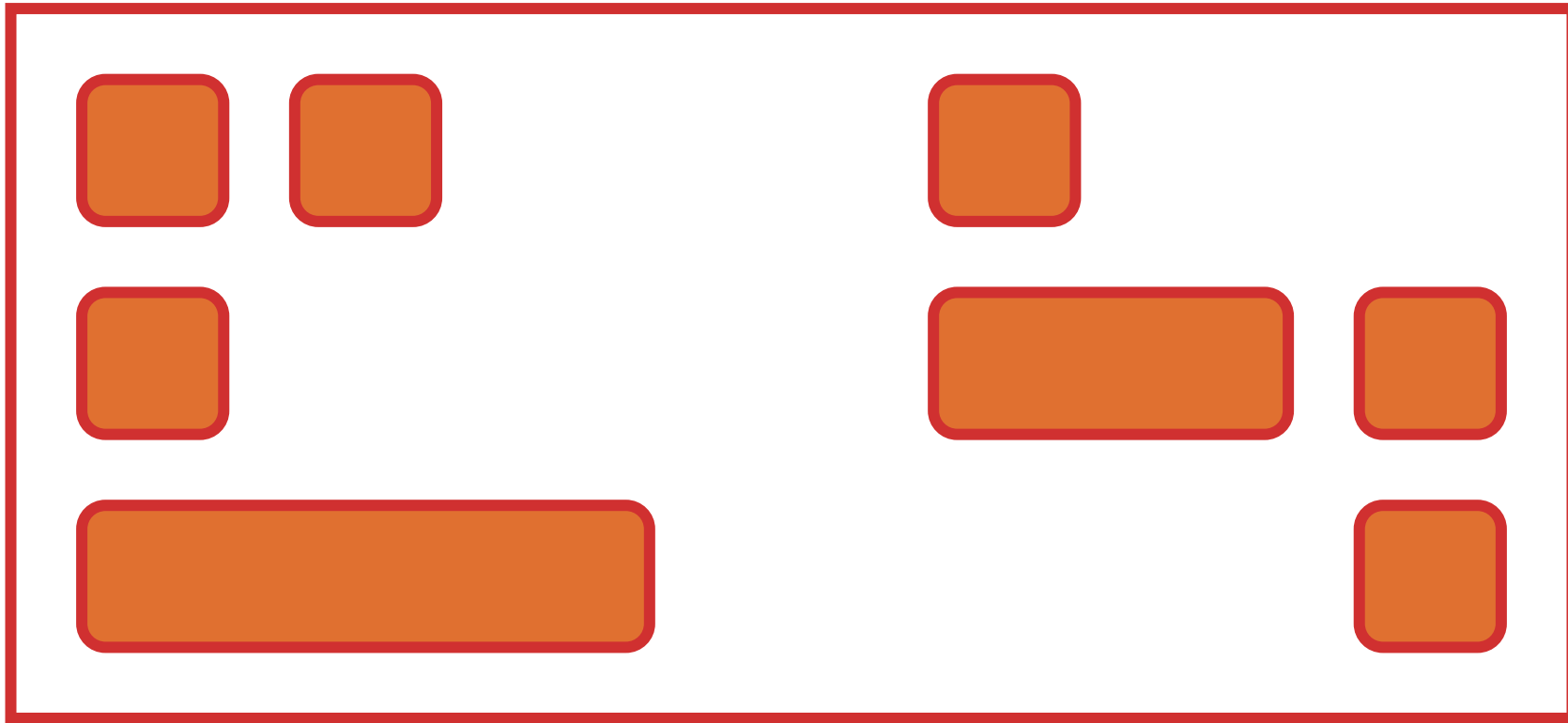


GC roots = Local variables + Static fields

Java 1



Java 1



Java 1

 It works. That's the good news

 It leaves holes everywhere

**It would be nice not to
have to walk through
every object every time**

The generational hypothesis

The generational hypothesis



Either objects die young



Or objects stick around for a long time (or forever)

Serial GC

Java 1.2 (1998)

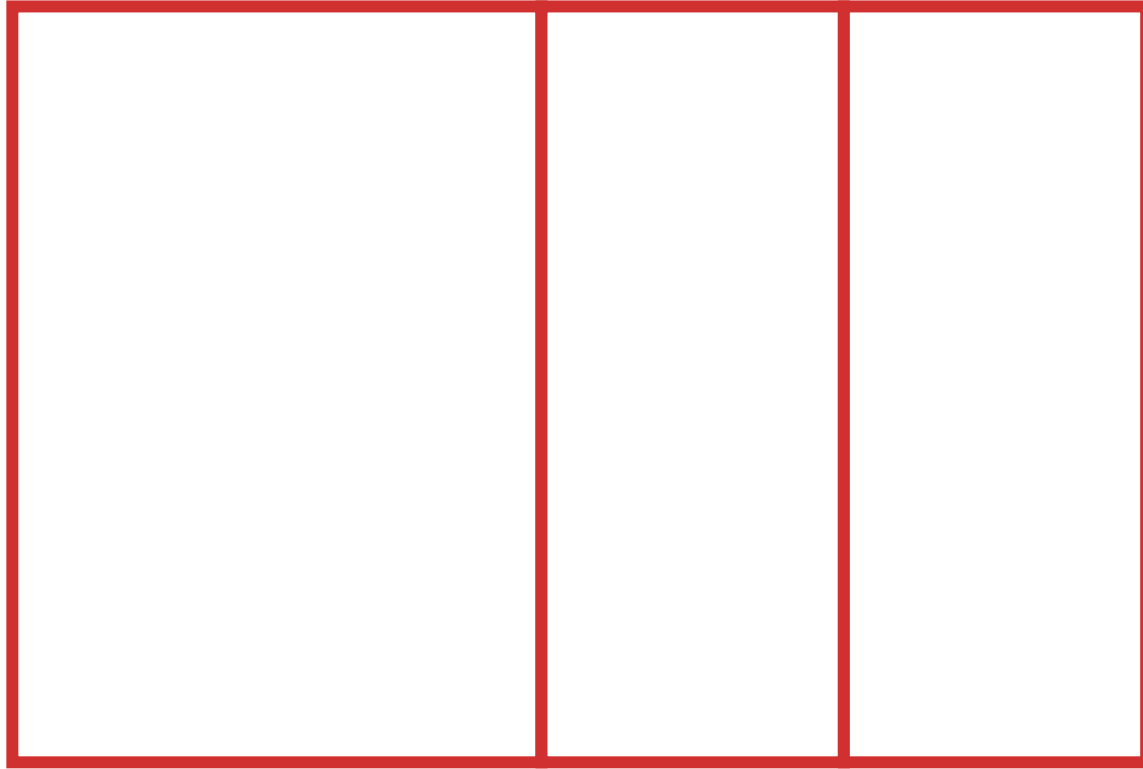
Serial GC

Survivor

Eden

S0

S1



Young

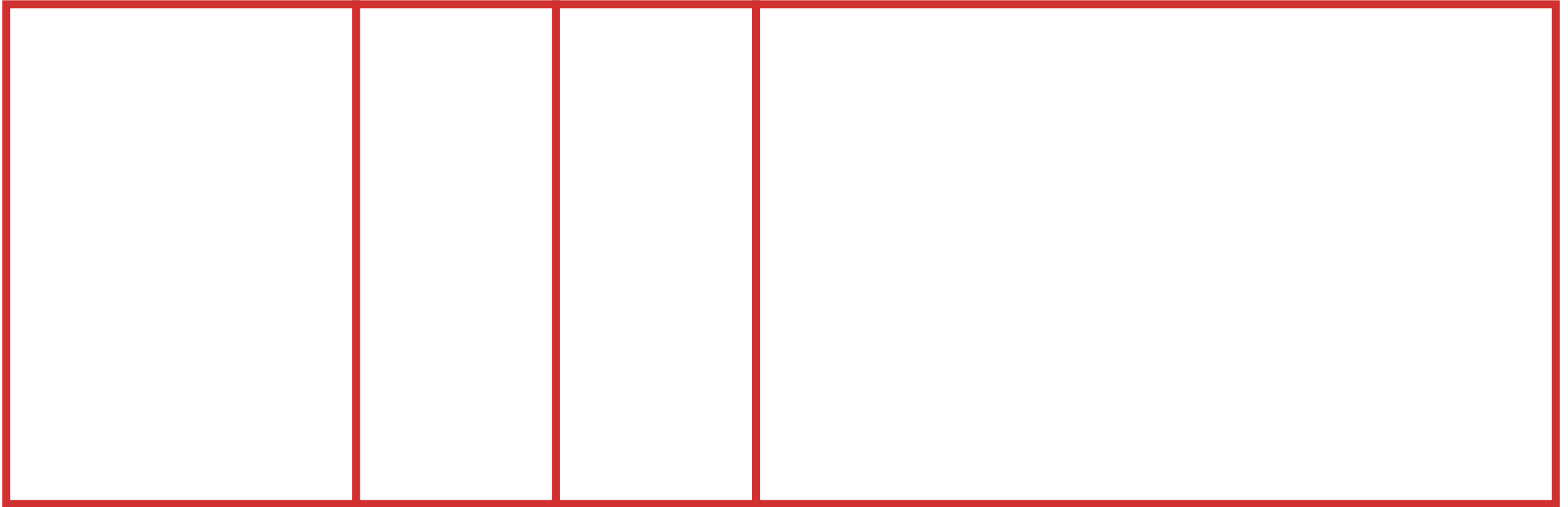
Serial GC

Eden

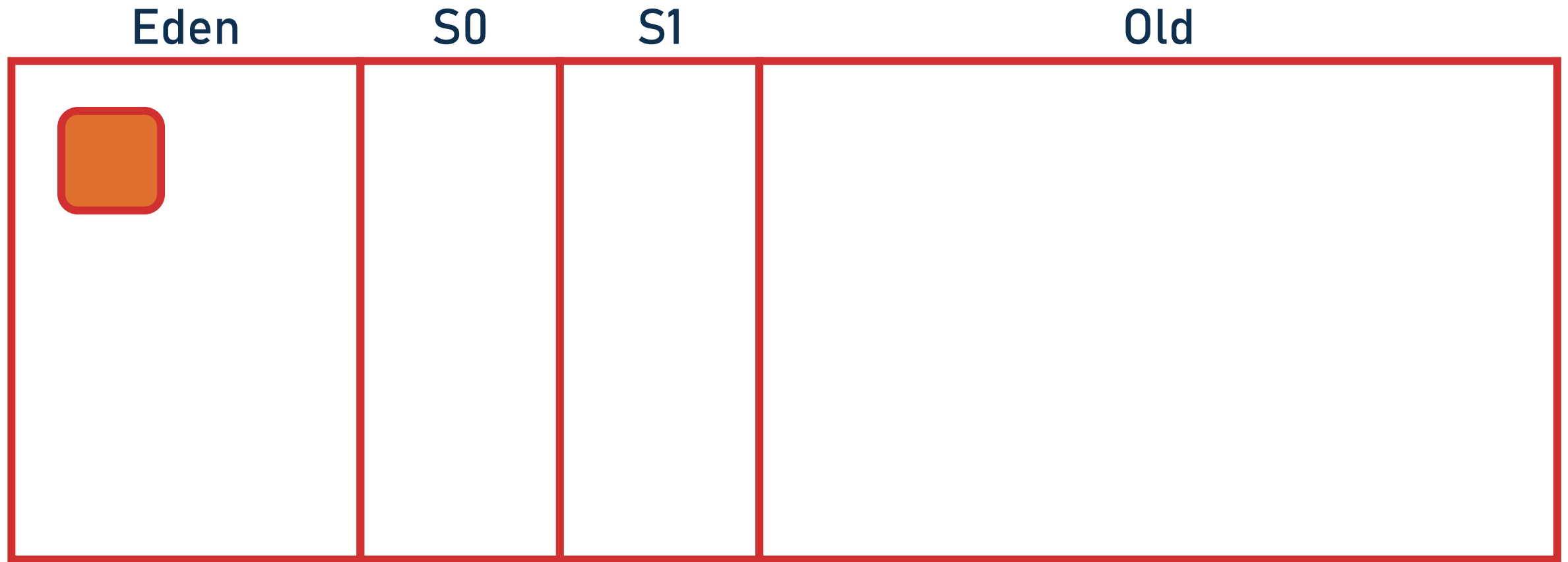
S0

S1

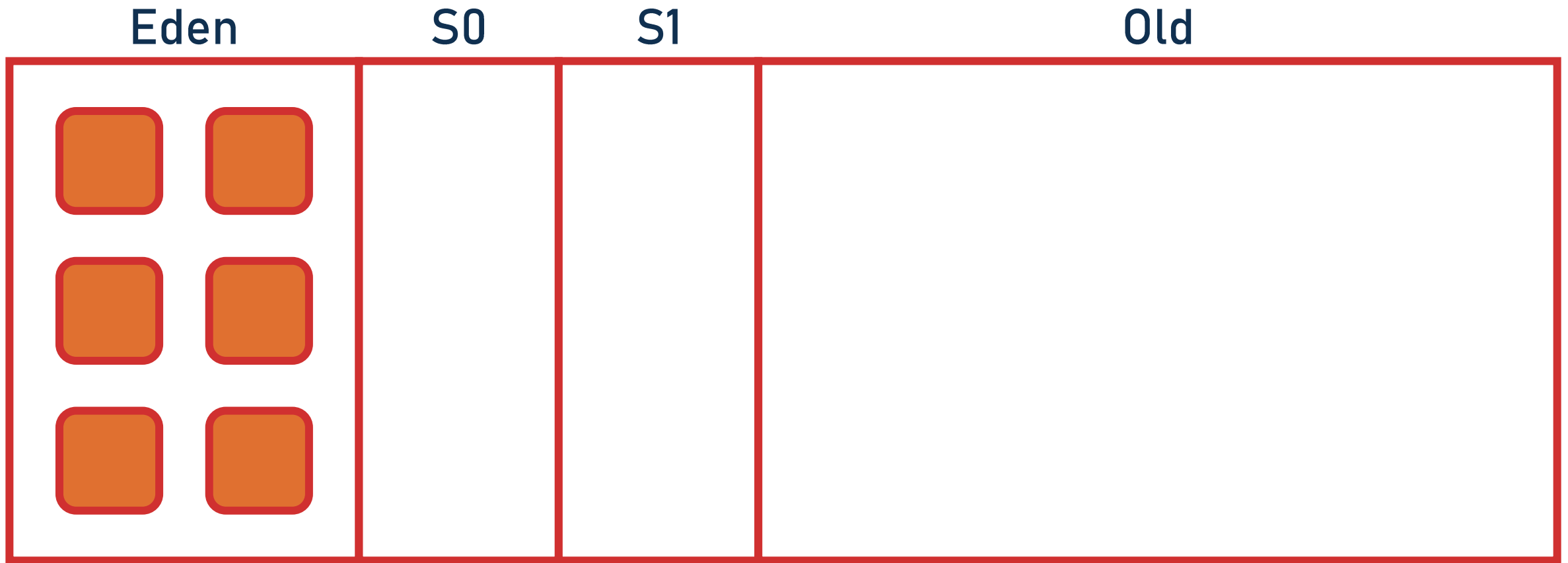
Old



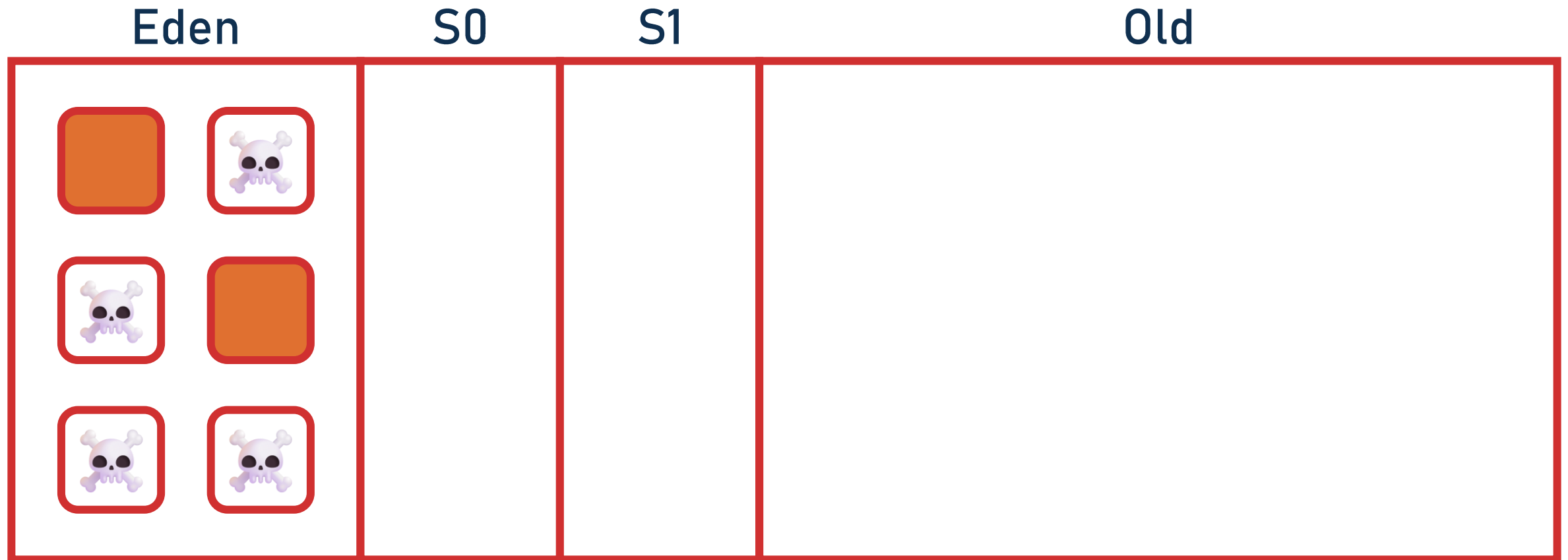
Serial GC



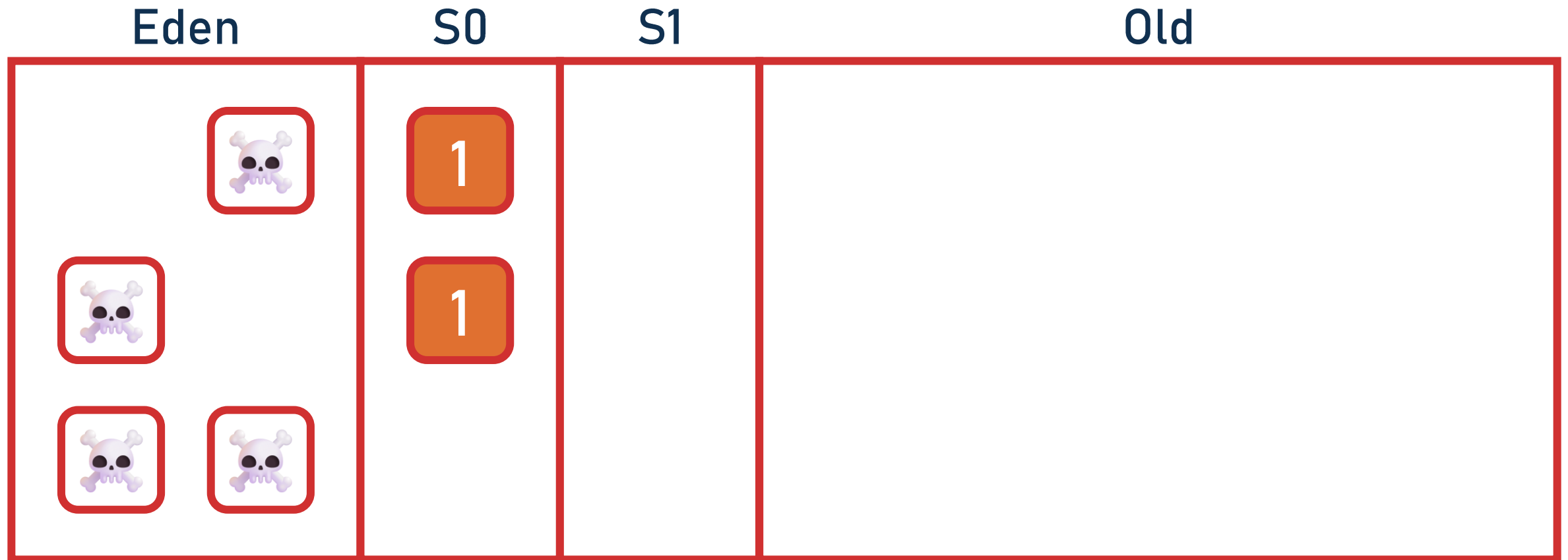
Serial GC



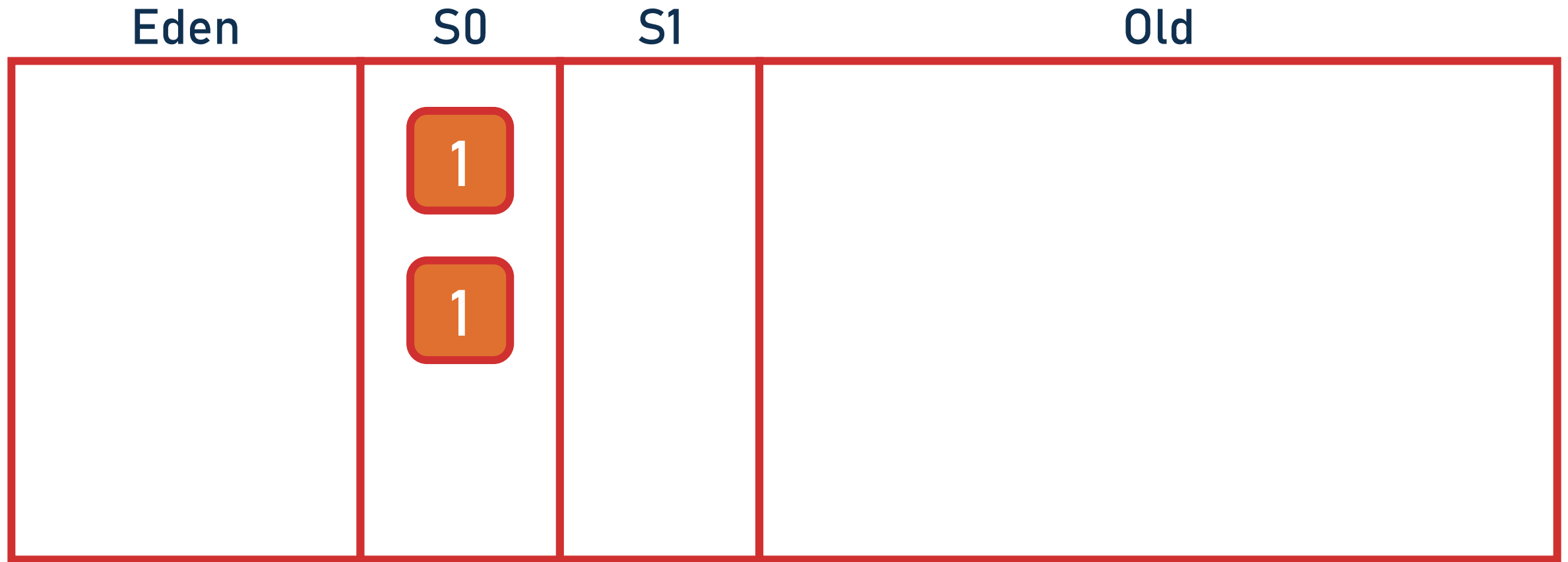
Serial GC



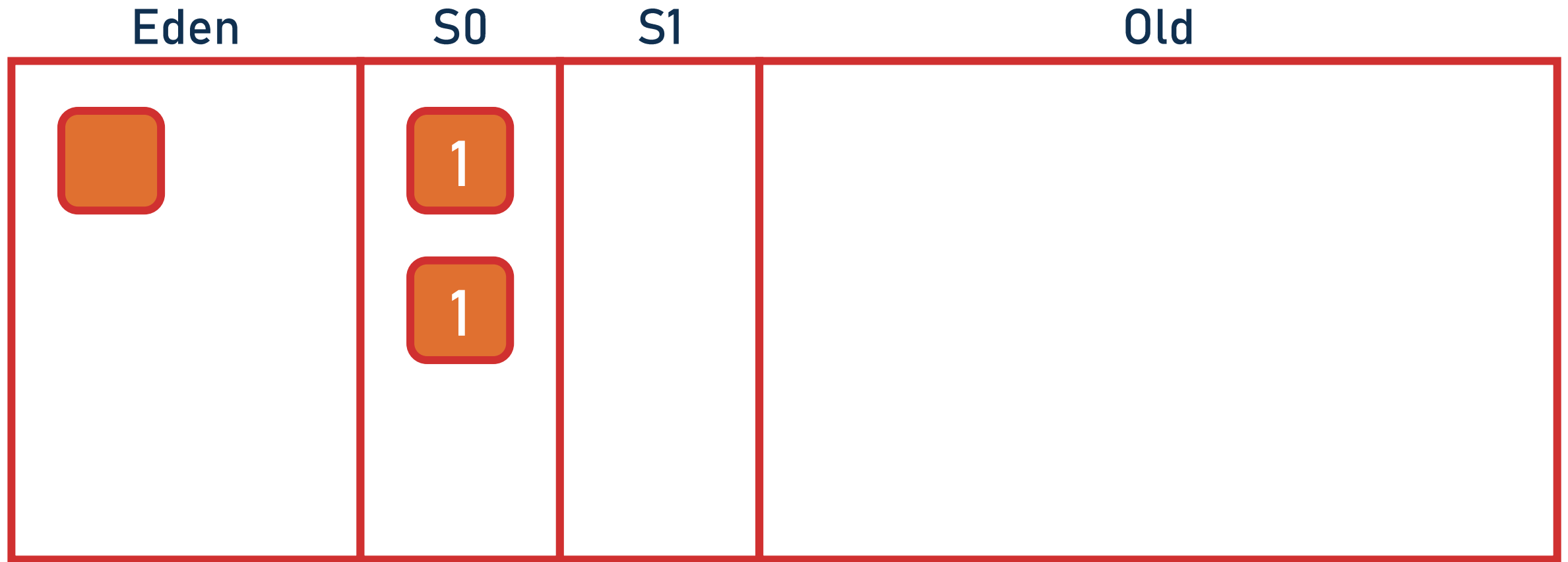
Serial GC



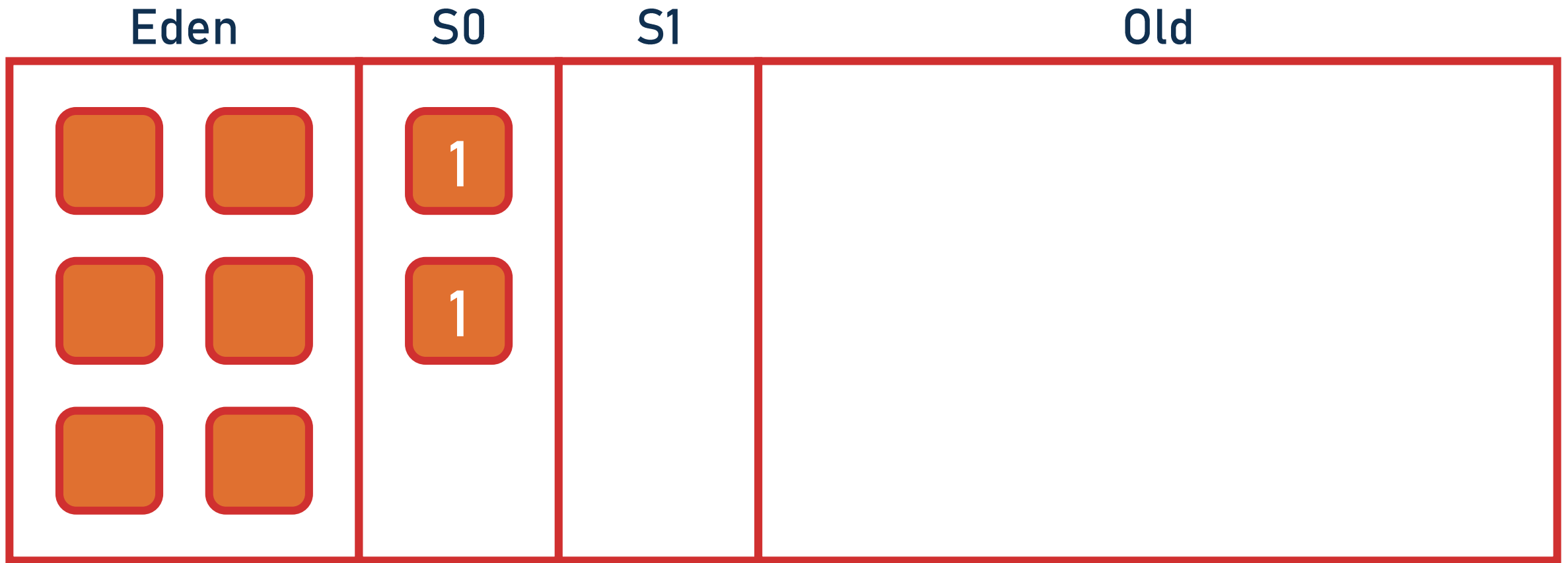
Serial GC



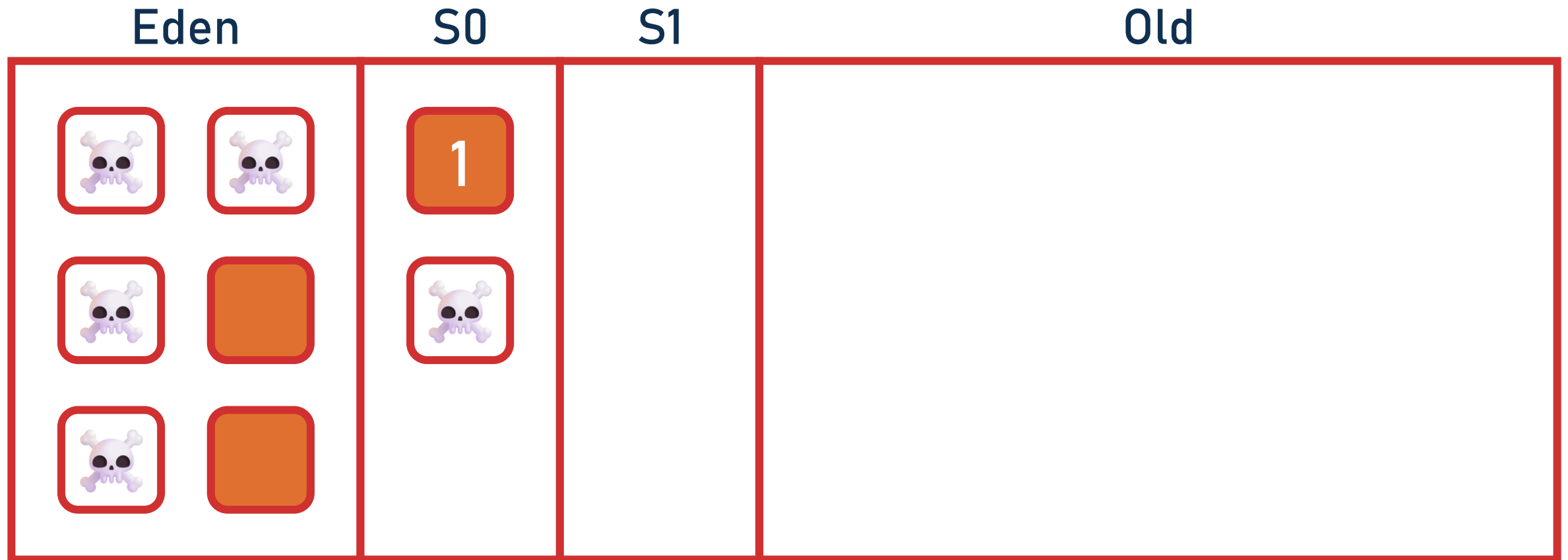
Serial GC



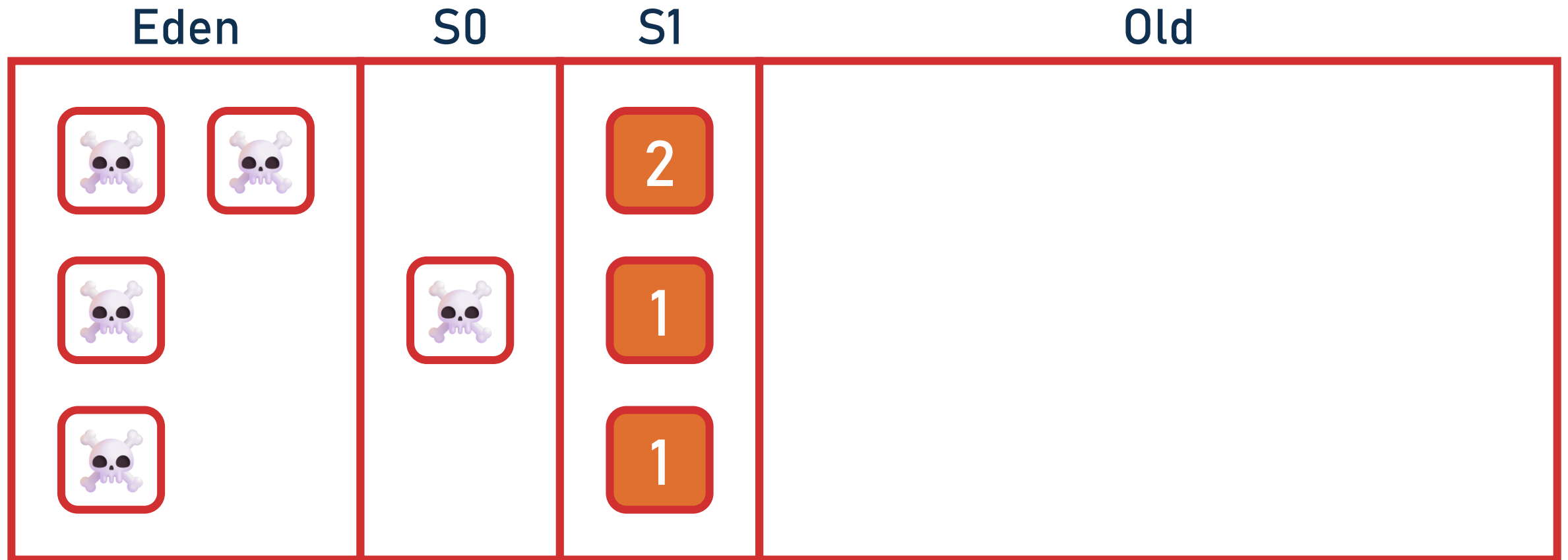
Serial GC



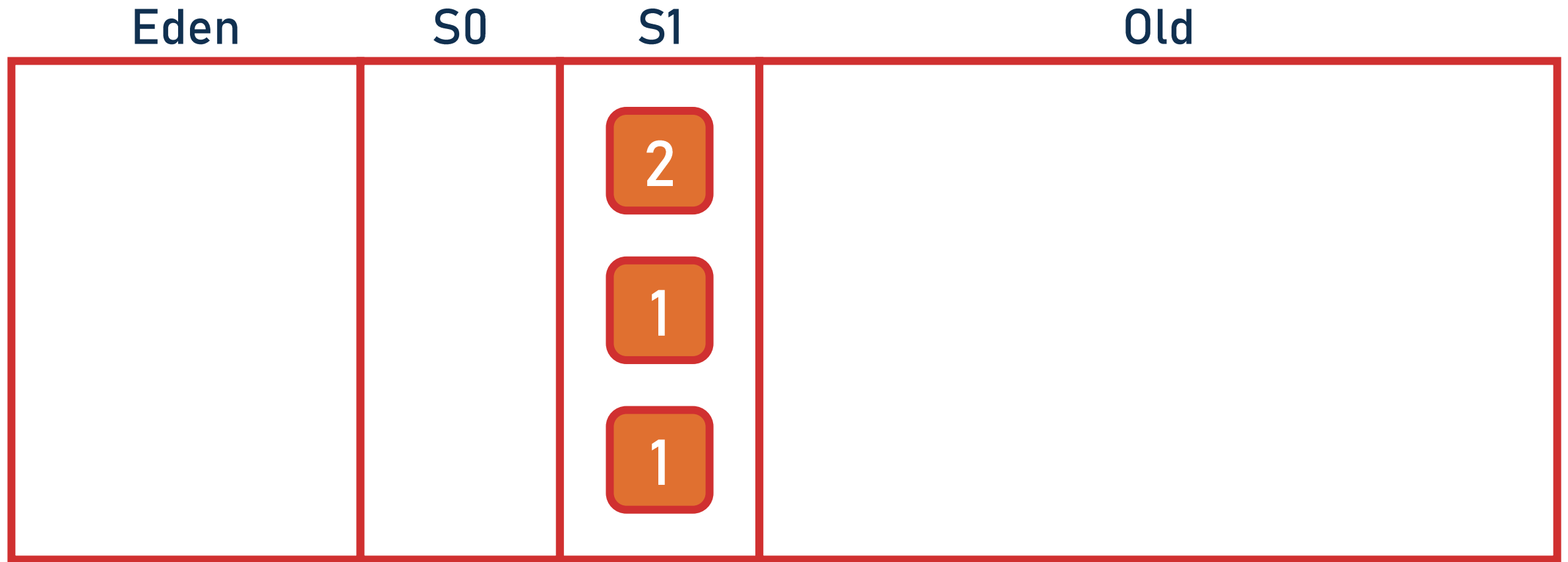
Serial GC



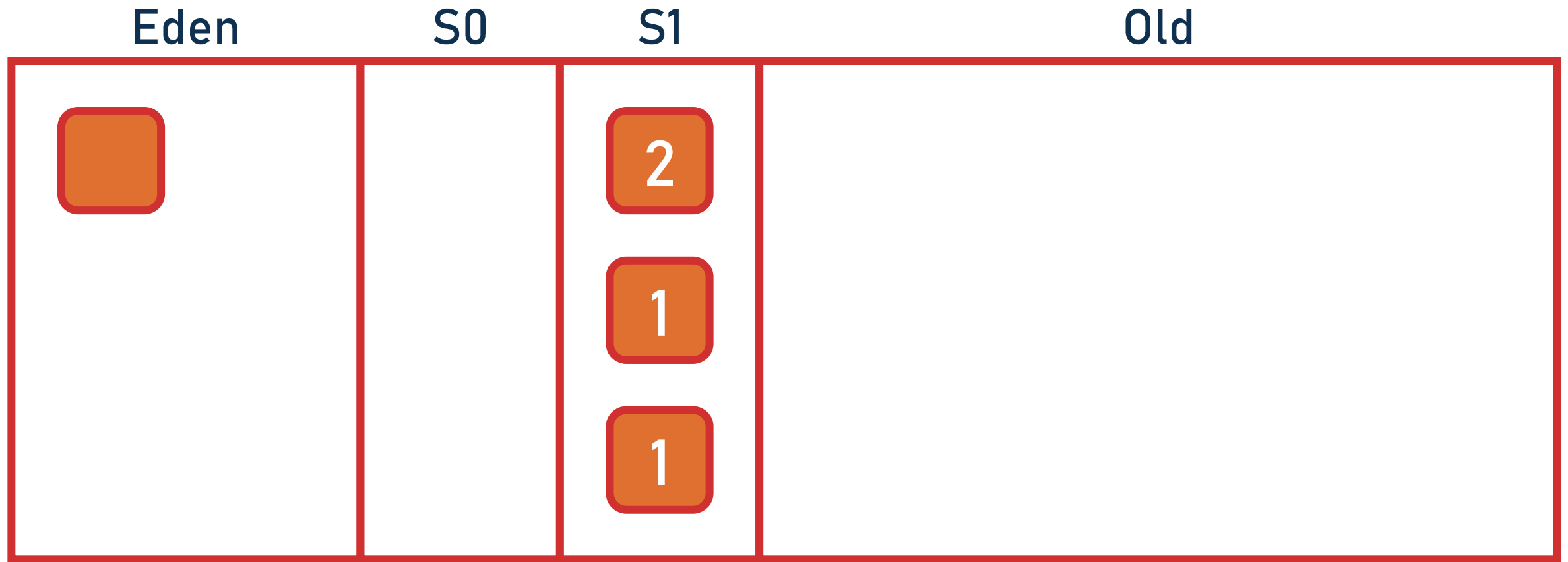
Serial GC



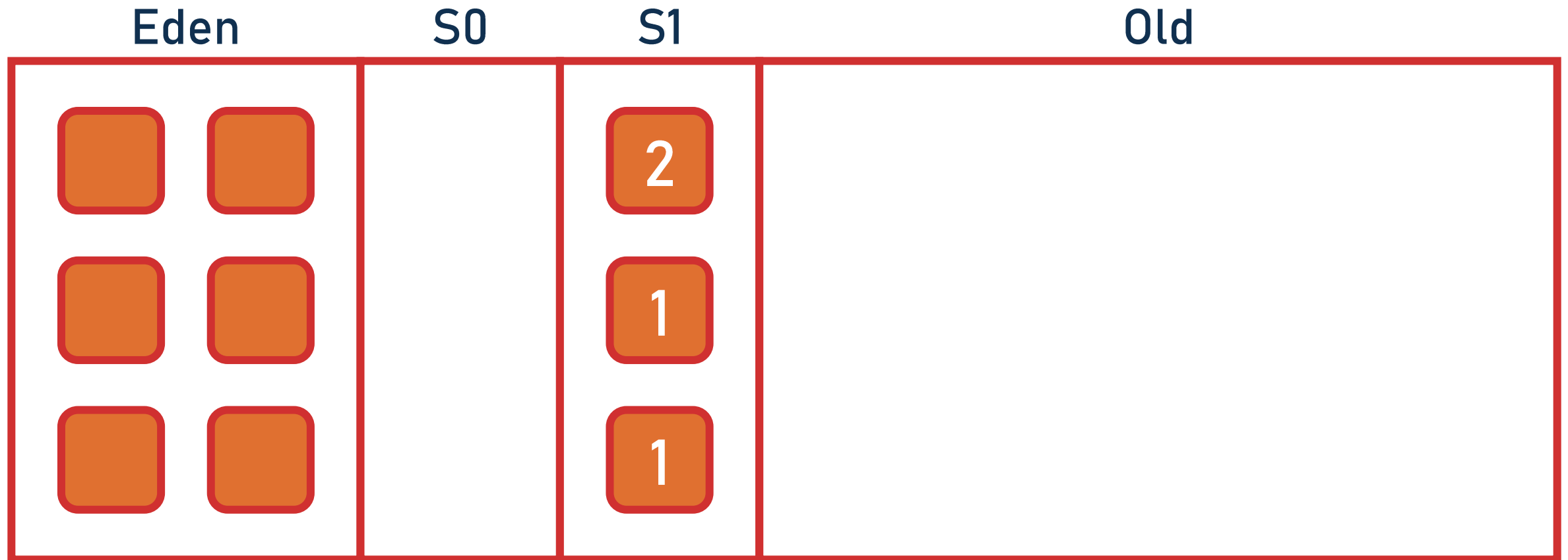
Serial GC



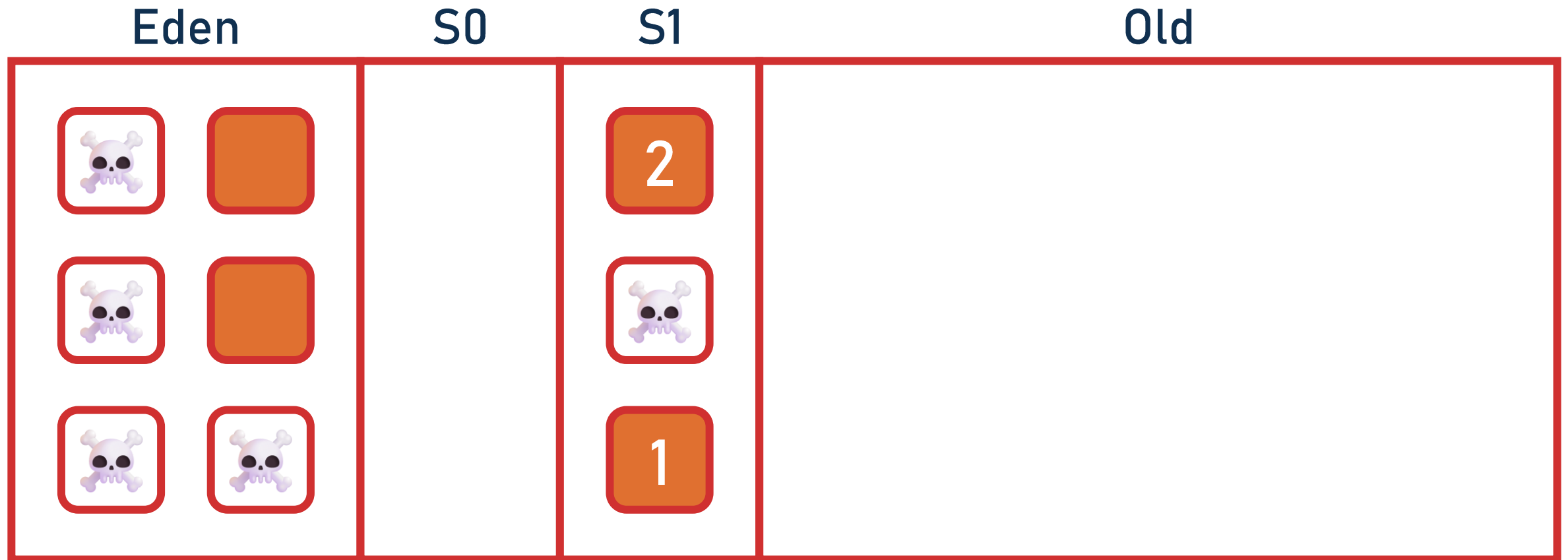
Serial GC



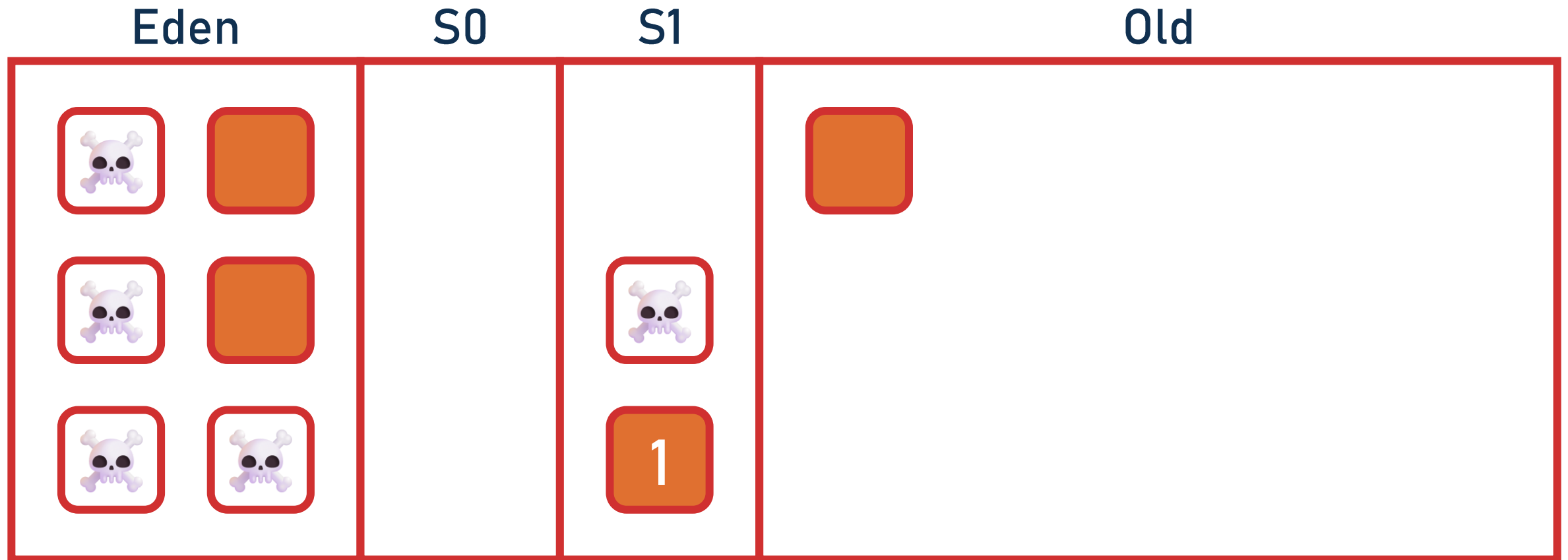
Serial GC



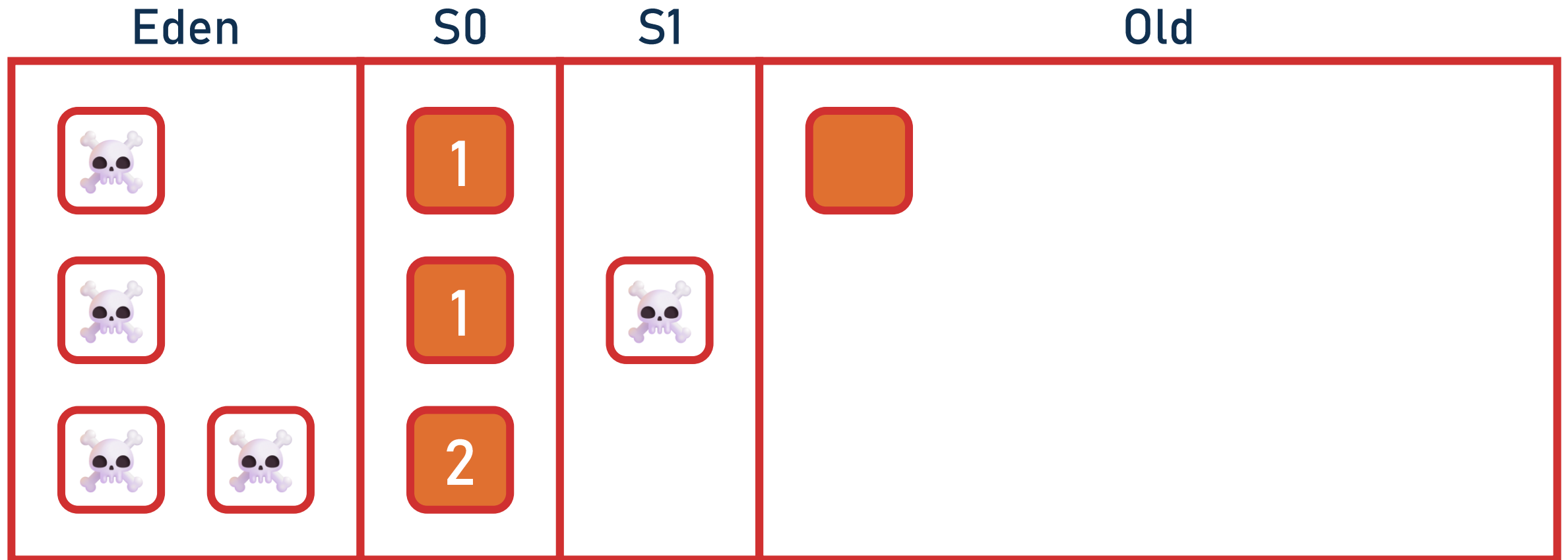
Serial GC



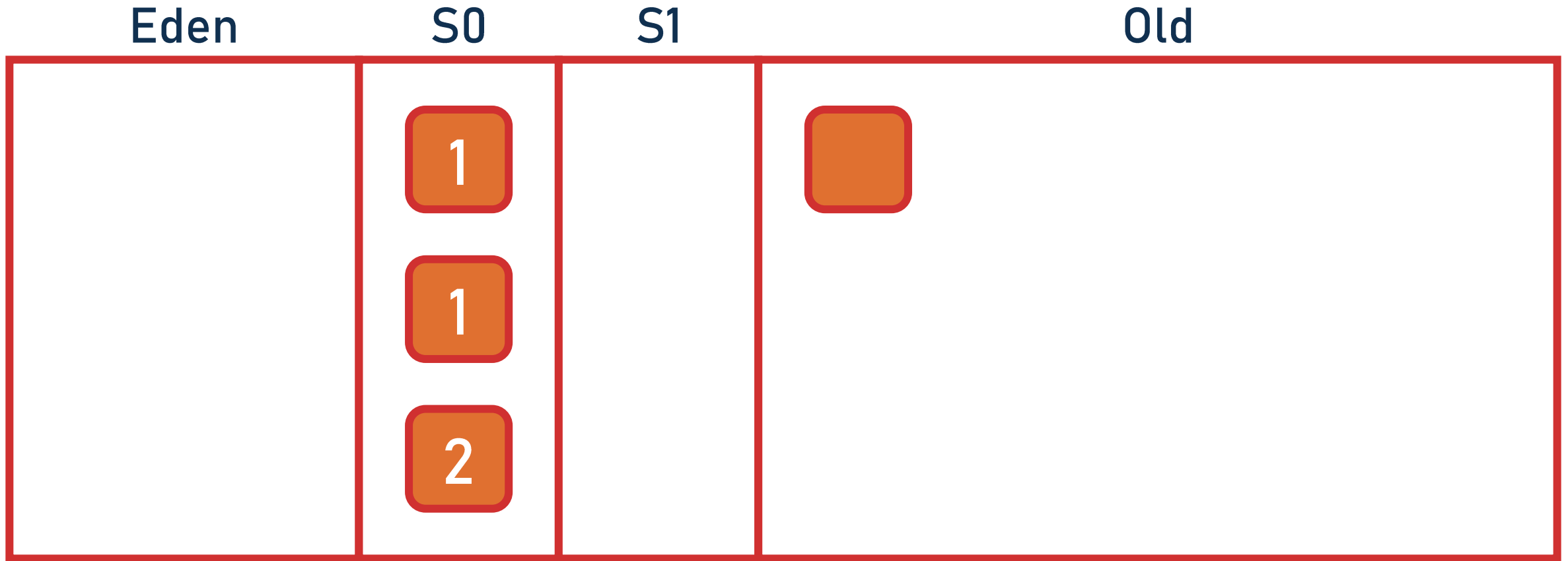
Serial GC



Serial GC

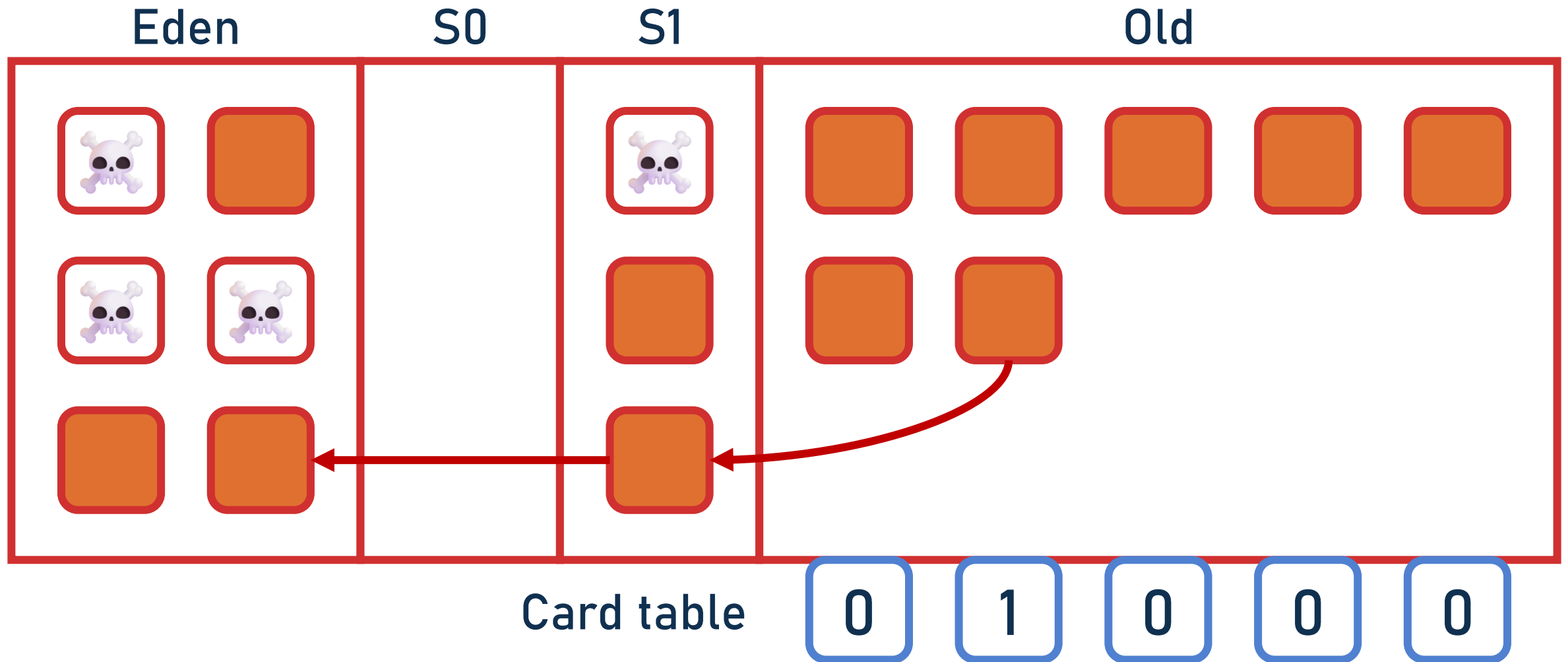


Serial GC



(and it keeps going)

Serial GC

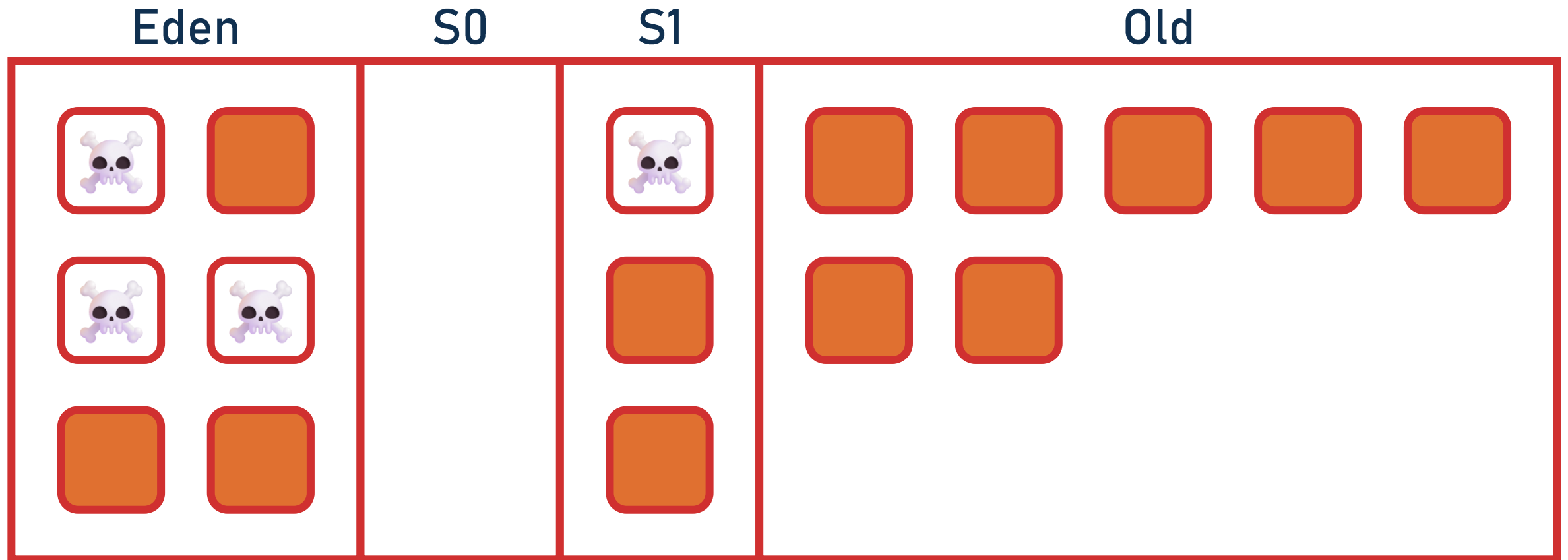


Serial GC – Write barrier

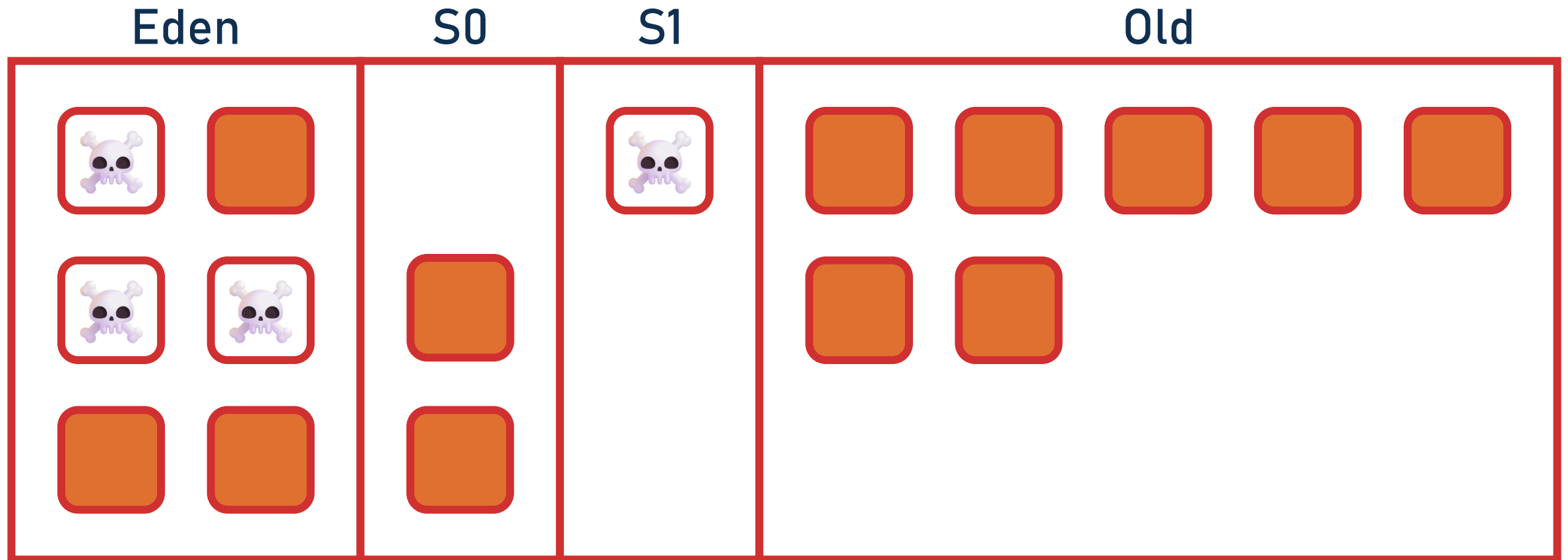
```
cache.addValue(value);

if (isOldObject(cache) && !isOldObject(value)) {
    int index = getMemoryAddress(cache) / 512;
    cardTable[index] = 1;
}
```

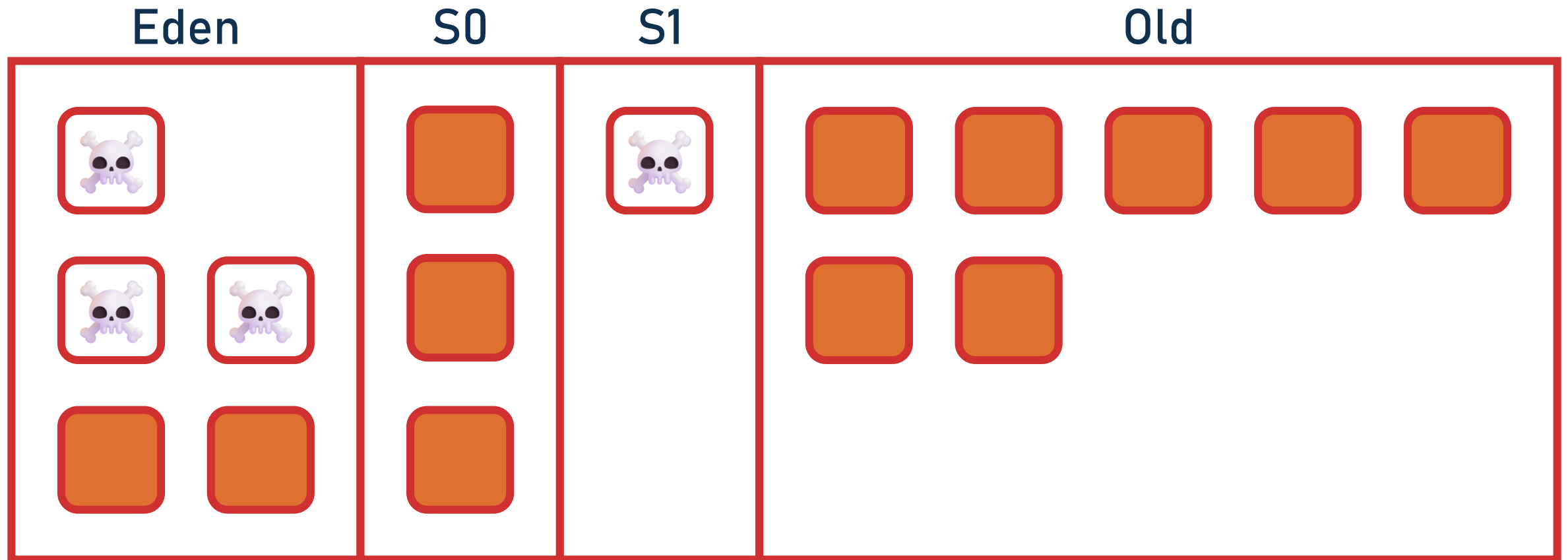
Serial GC



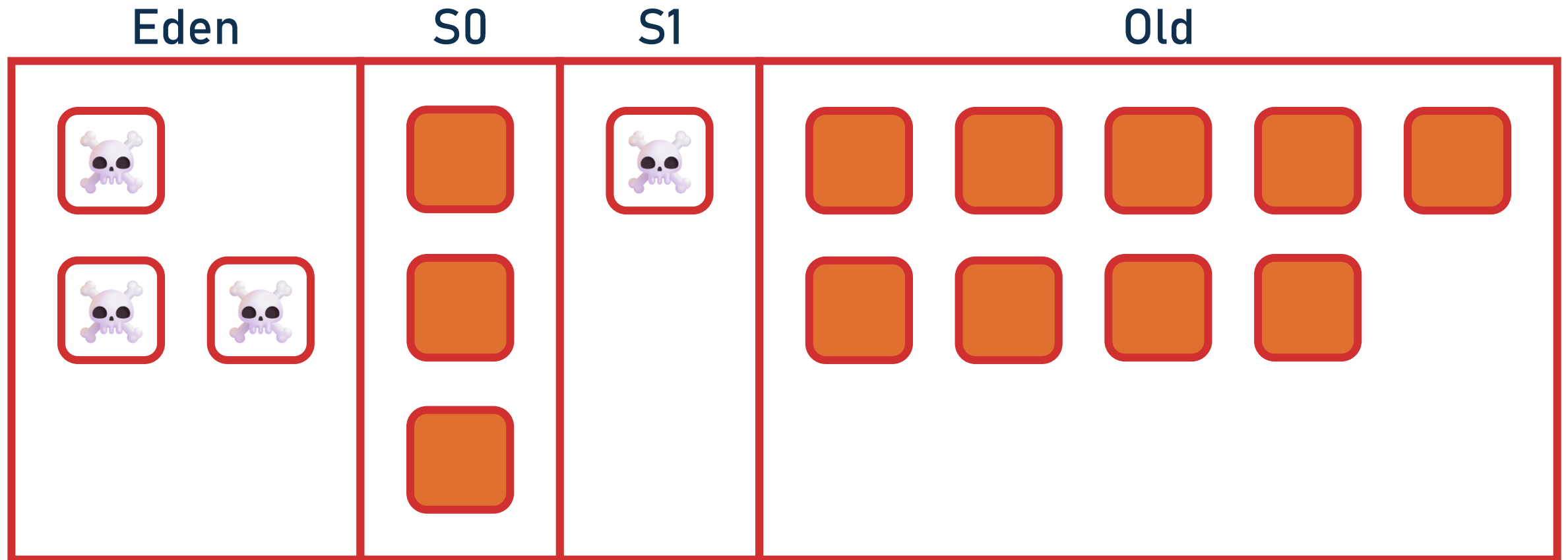
Serial GC



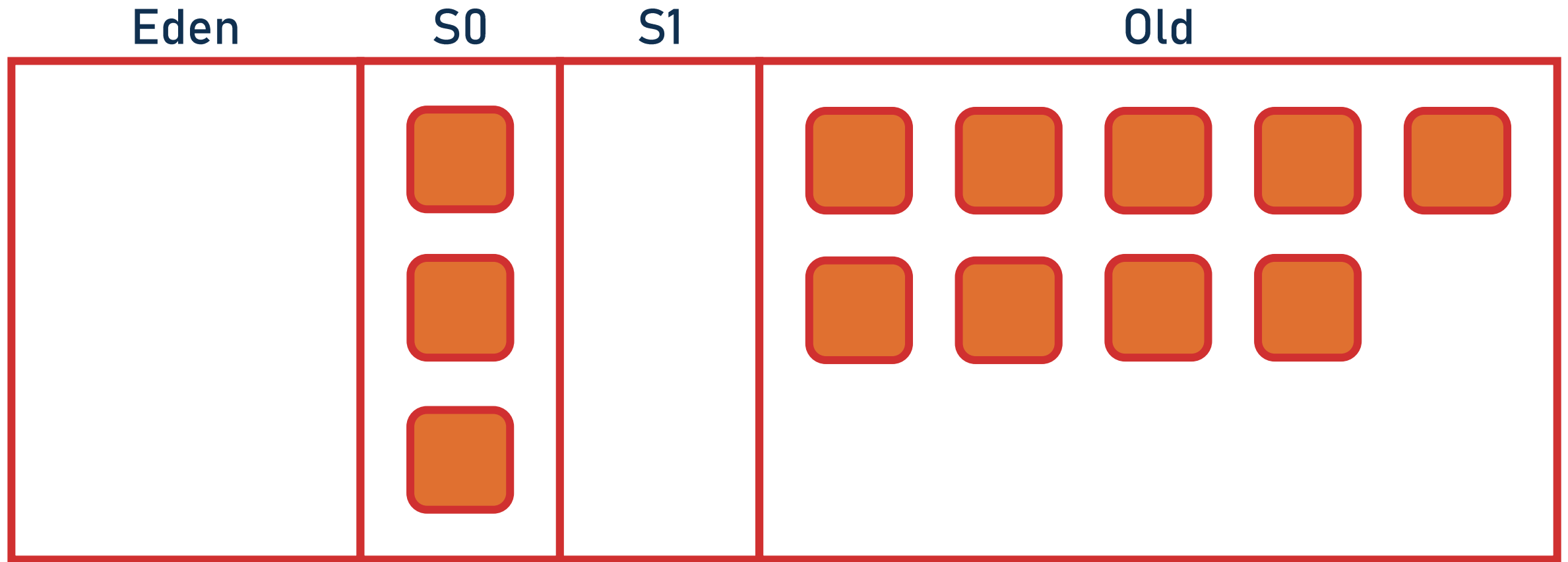
Serial GC



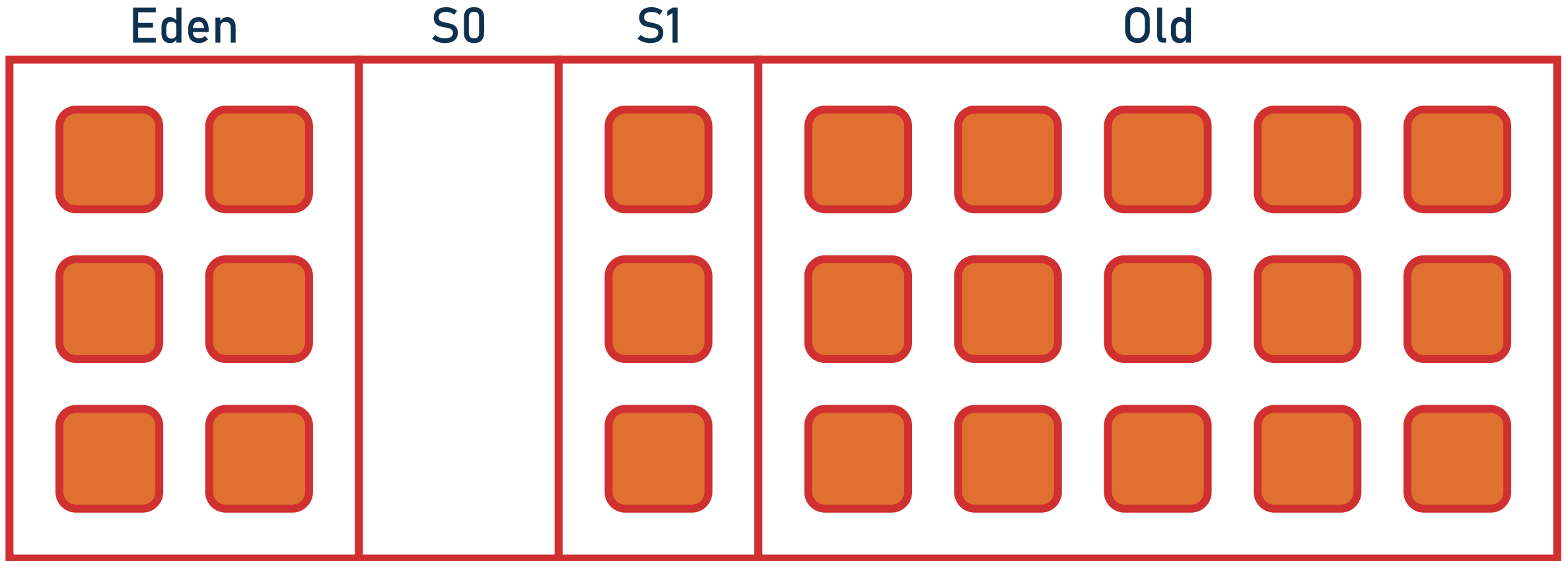
Serial GC



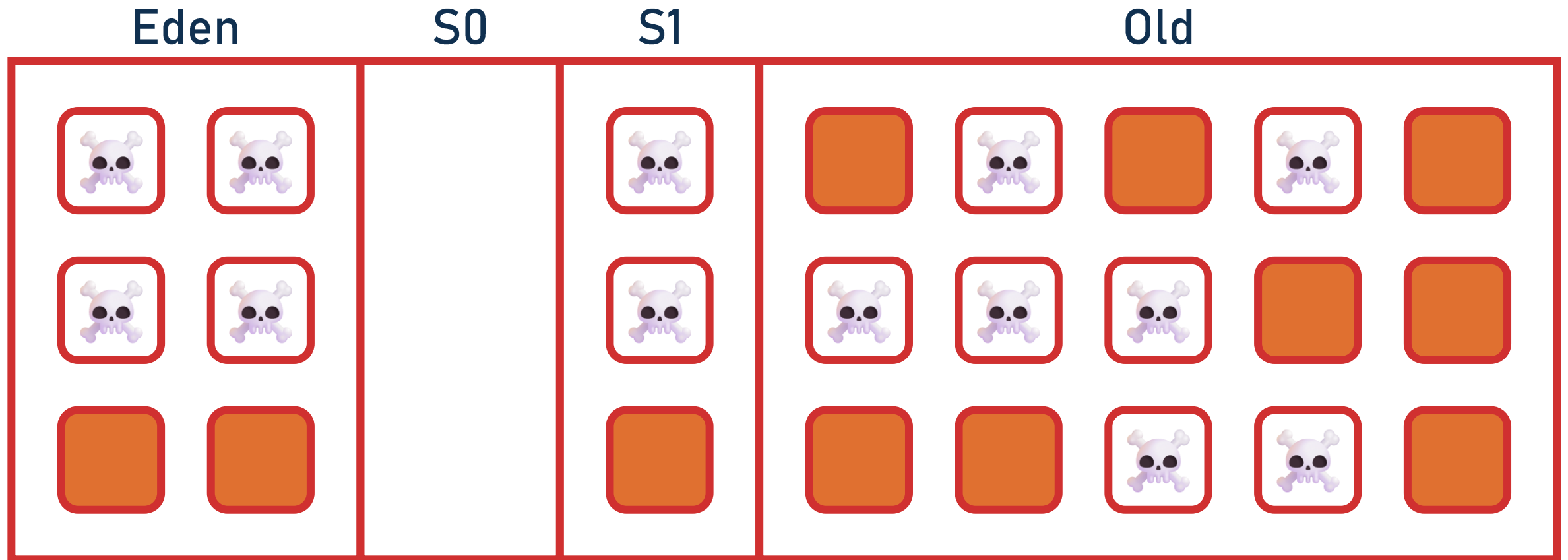
Serial GC



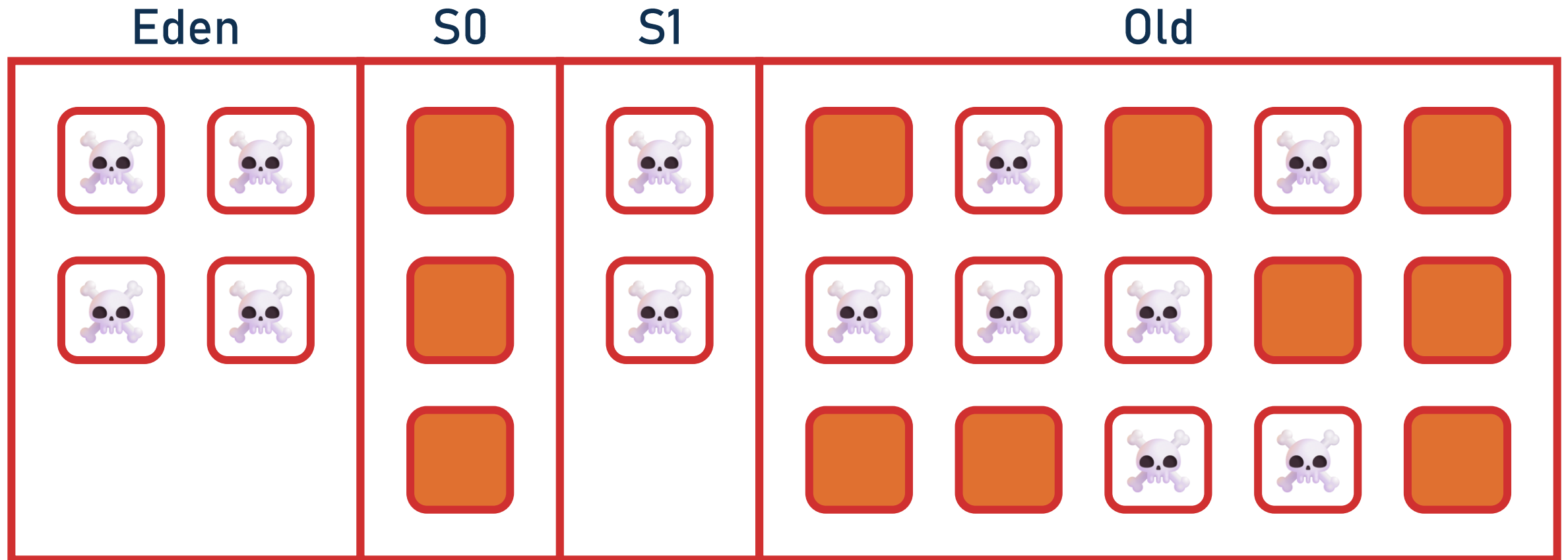
Serial GC – Full GC



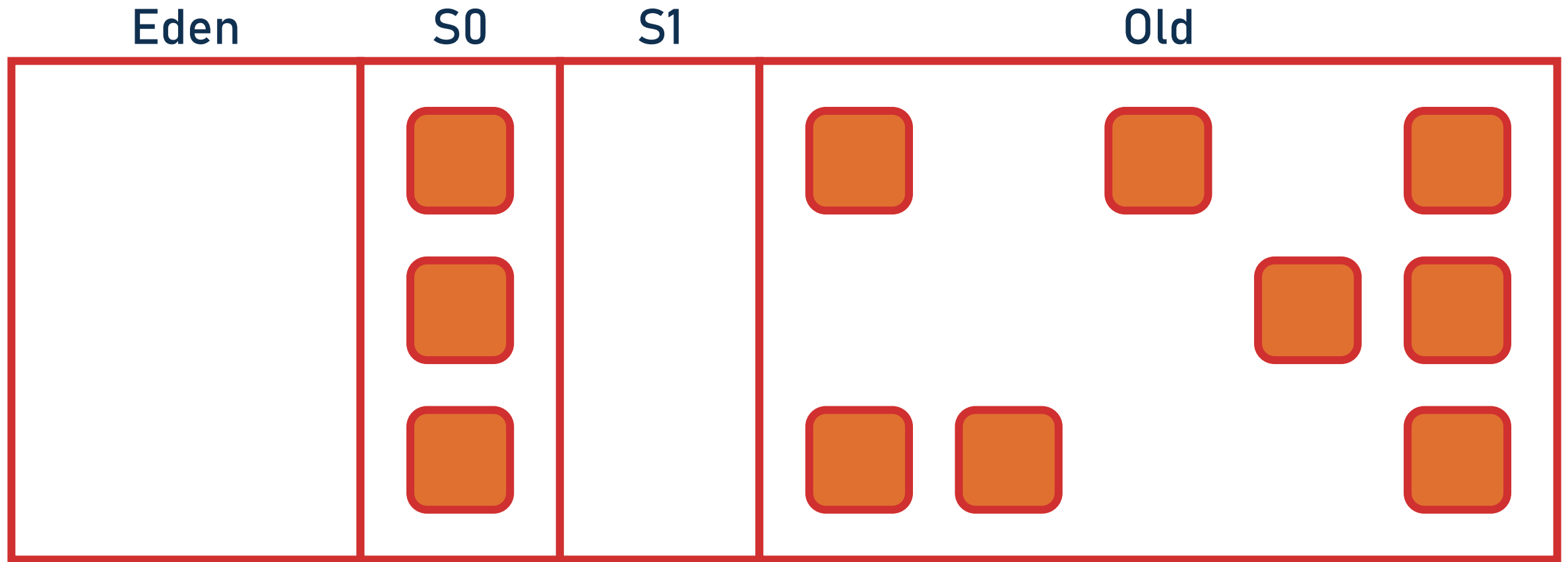
Serial GC – Full GC



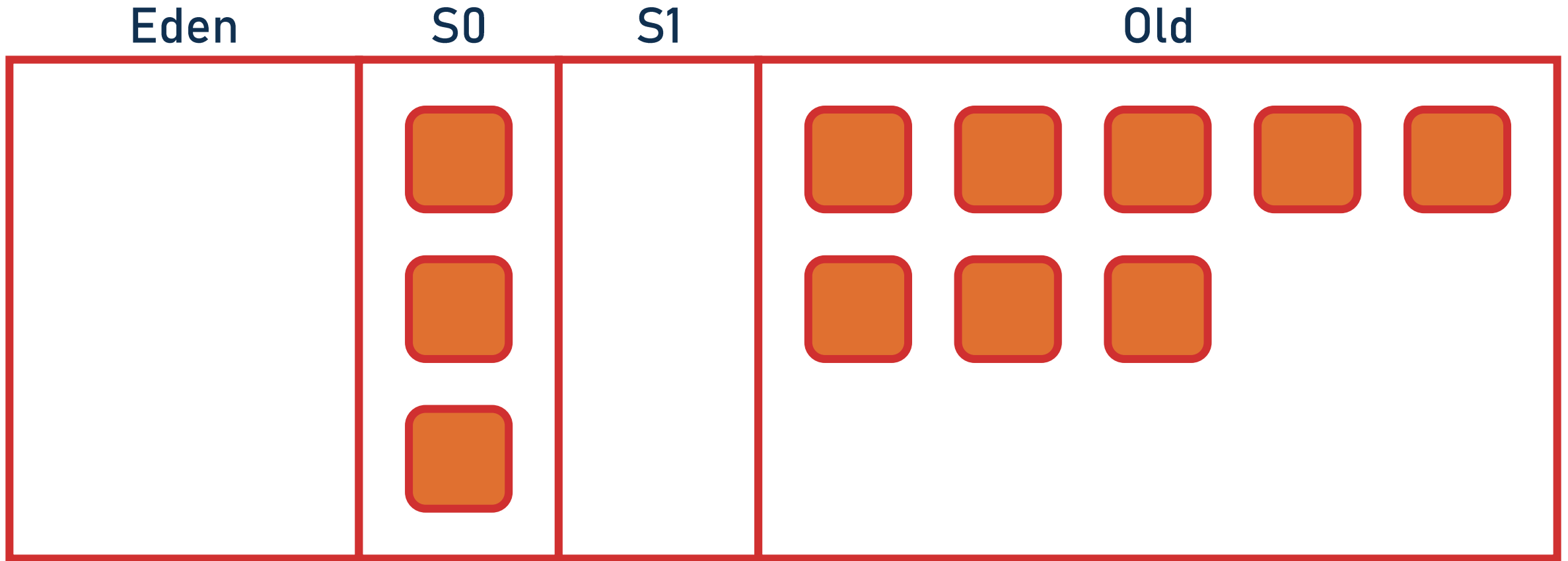
Serial GC – Full GC



Serial GC – Full GC



Serial GC – Full GC



Serial GC

- 💖 Still around and still maintained
- ➖ Stop-the-world
- 👉 Single-threaded
- 🎯 The GC with the lowest overhead
- 👉 The default GC when < 2 GB of RAM or < 2 CPUs
- 🐜 Ideal for microservices

Parallel GC

Java 5 (2004)

Parallel GC

-  Same thing, but in parallel
-  Still stop-the-world
-  Pauses of several seconds on large heaps
-  The GC with the highest throughput
-  Ideal for data processing, batch jobs, etc.

Latency?

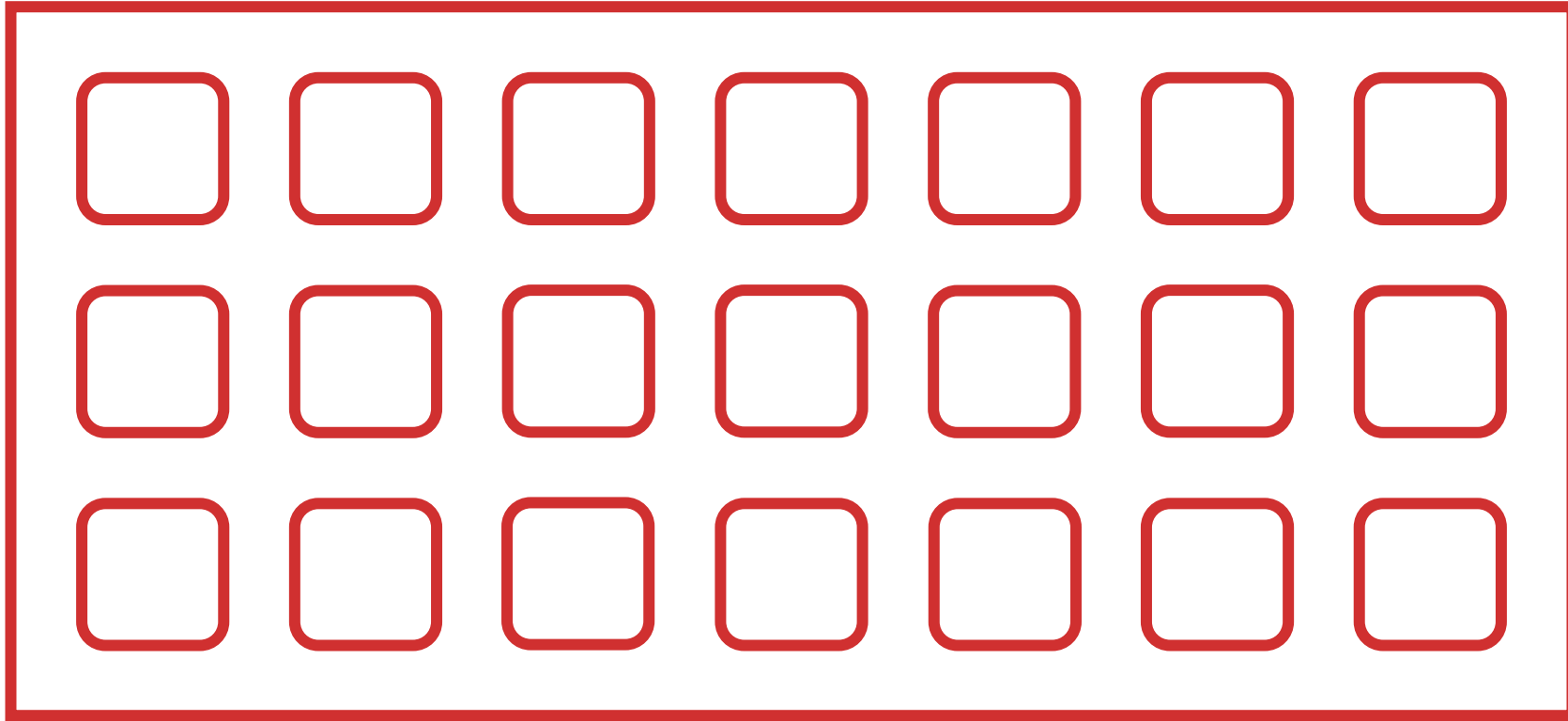
What about latency?

-  The GC can block the application for several seconds
-  Failed attempt: Train GC
-  Half-failed attempt: CMS GC
-  Successful attempt: G1 GC in Java 7 (default in Java 9)

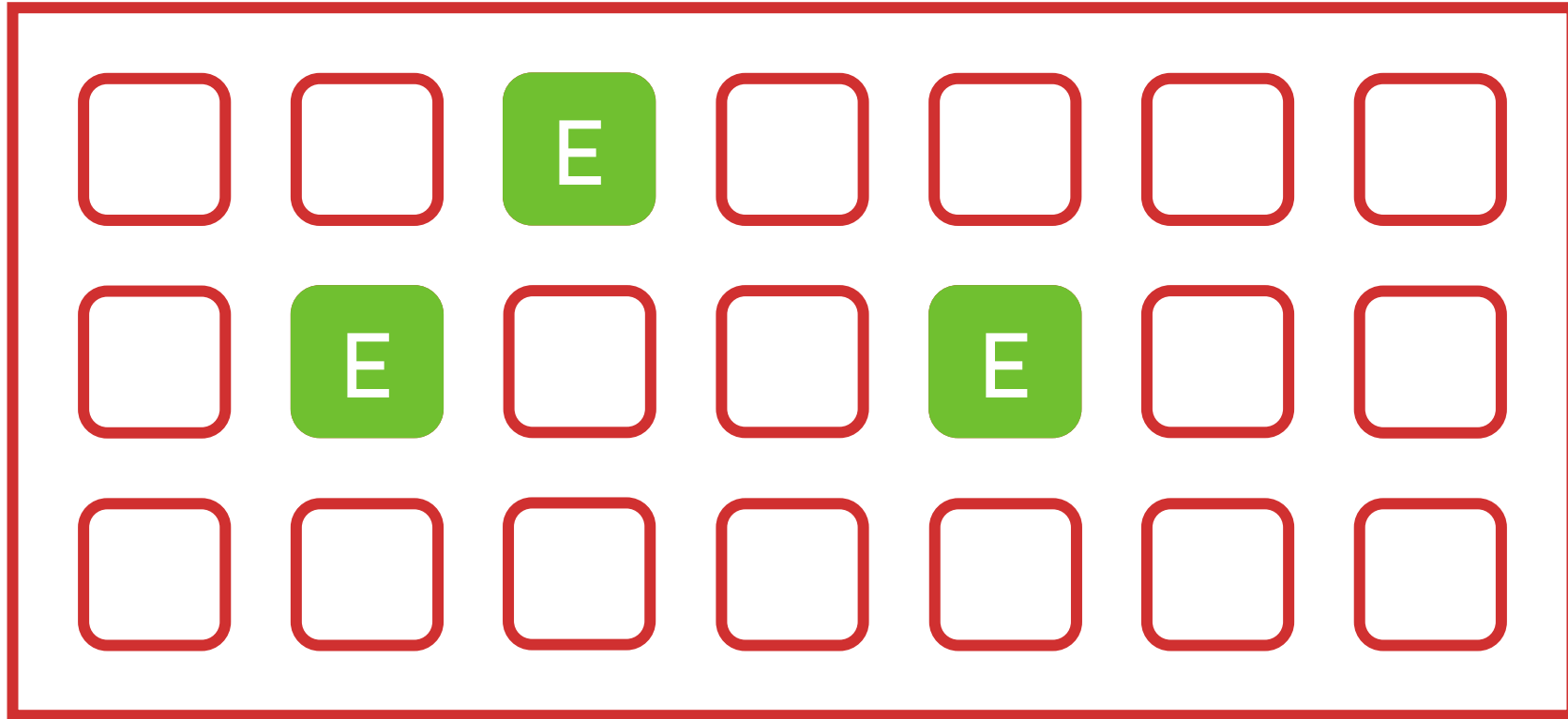
G1 GC

Java 7 (2011)

G1 GC

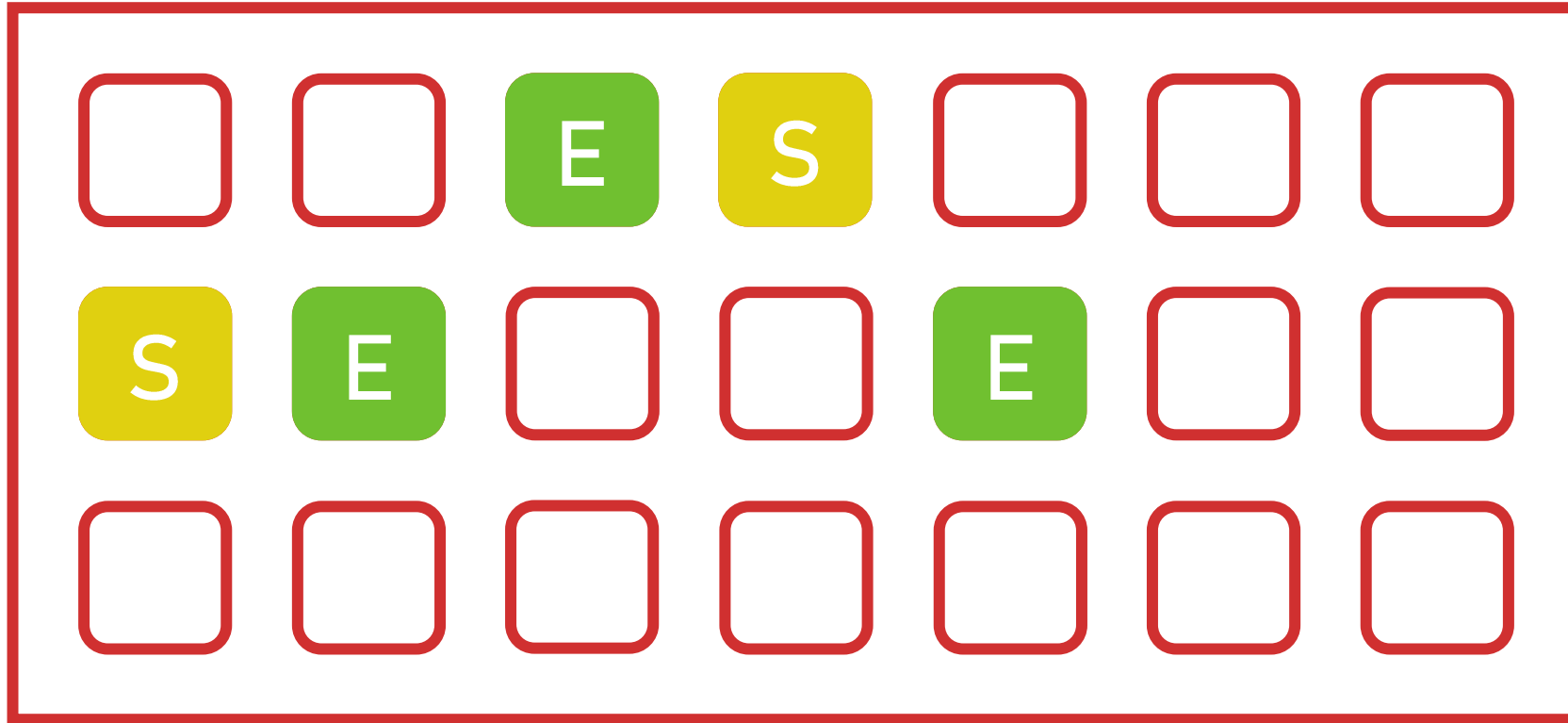


G1 GC



E Eden

G1 GC



Eden



Survivor

G1 GC



E Eden

S Survivor

0 Old

G1 GC



Eden



Survivor



Old



Humongous

G1 GC – Young collection



Eden



Survivor



Old



Humongous

G1 GC – Young collection



Eden



Survivor

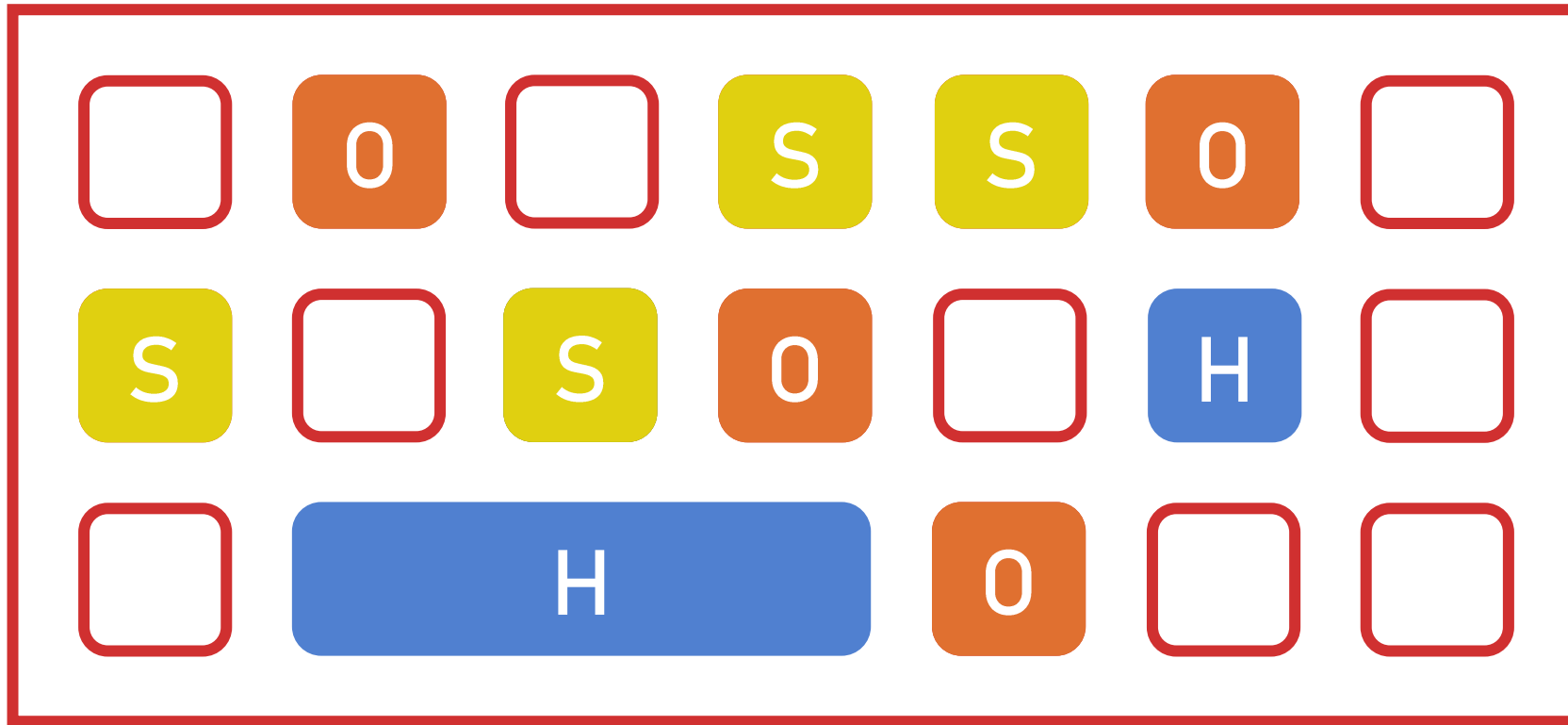


Old



Humongous

G1 GC – Young collection



Eden



Survivor

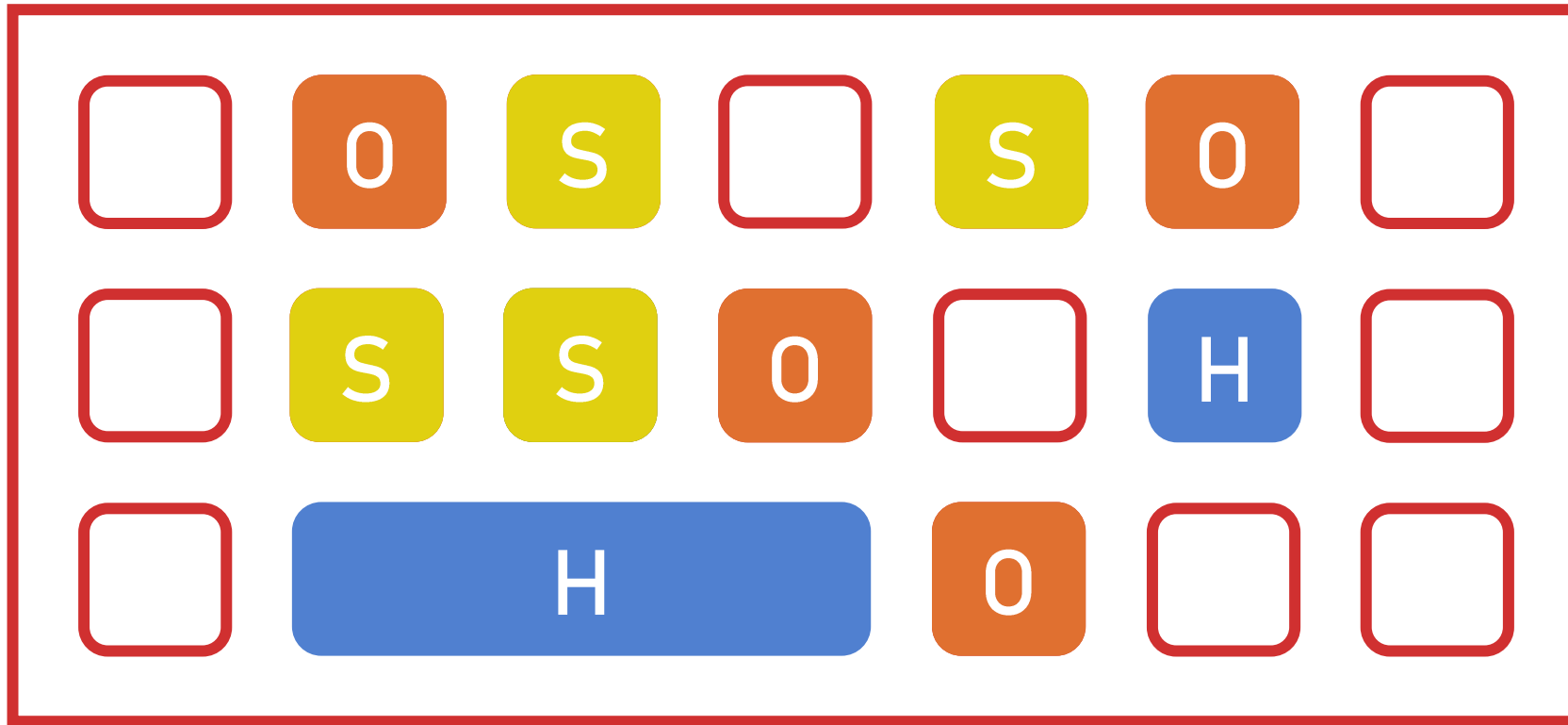


Old



Humongous

G1 GC – Young collection



Eden



Survivor

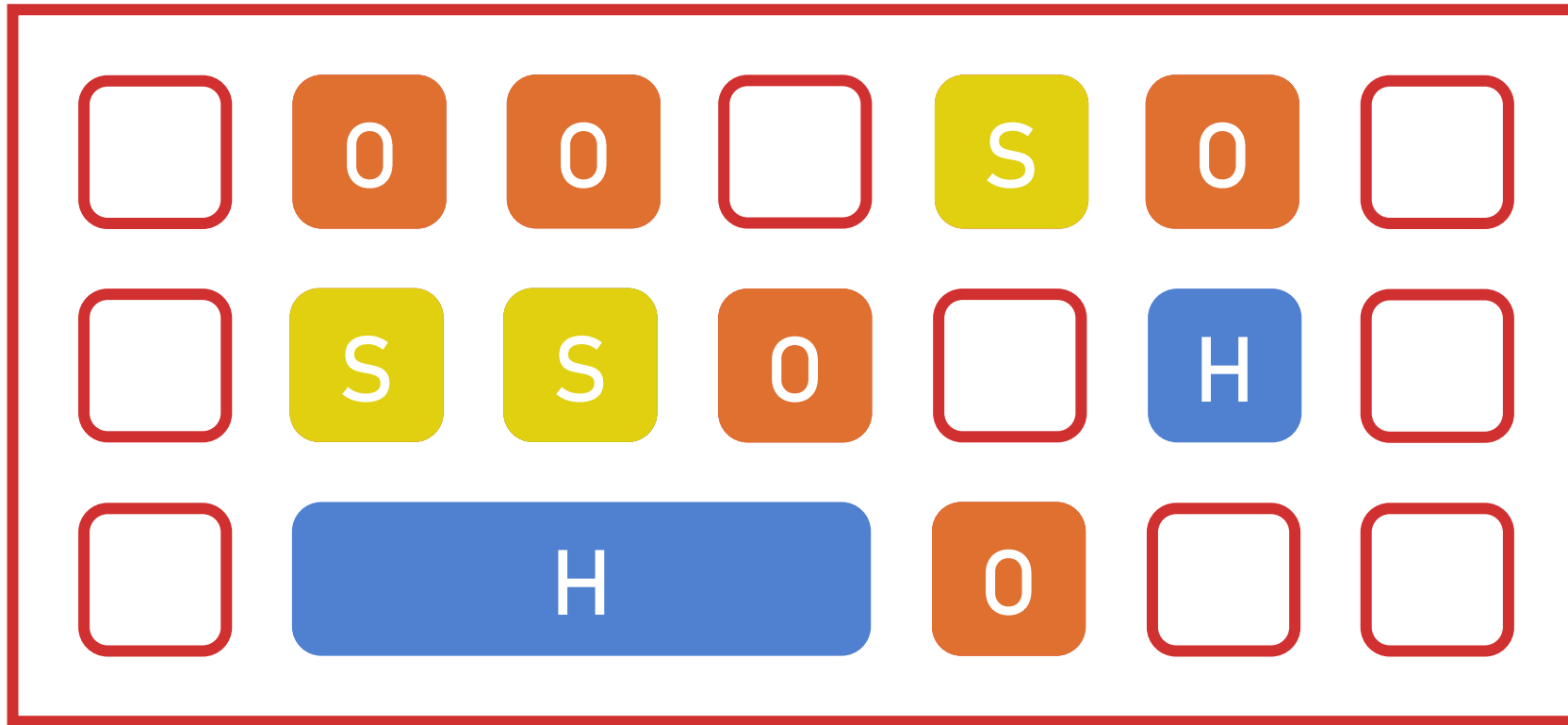


Old



Humongous

G1 GC – Young collection



Eden



Survivor

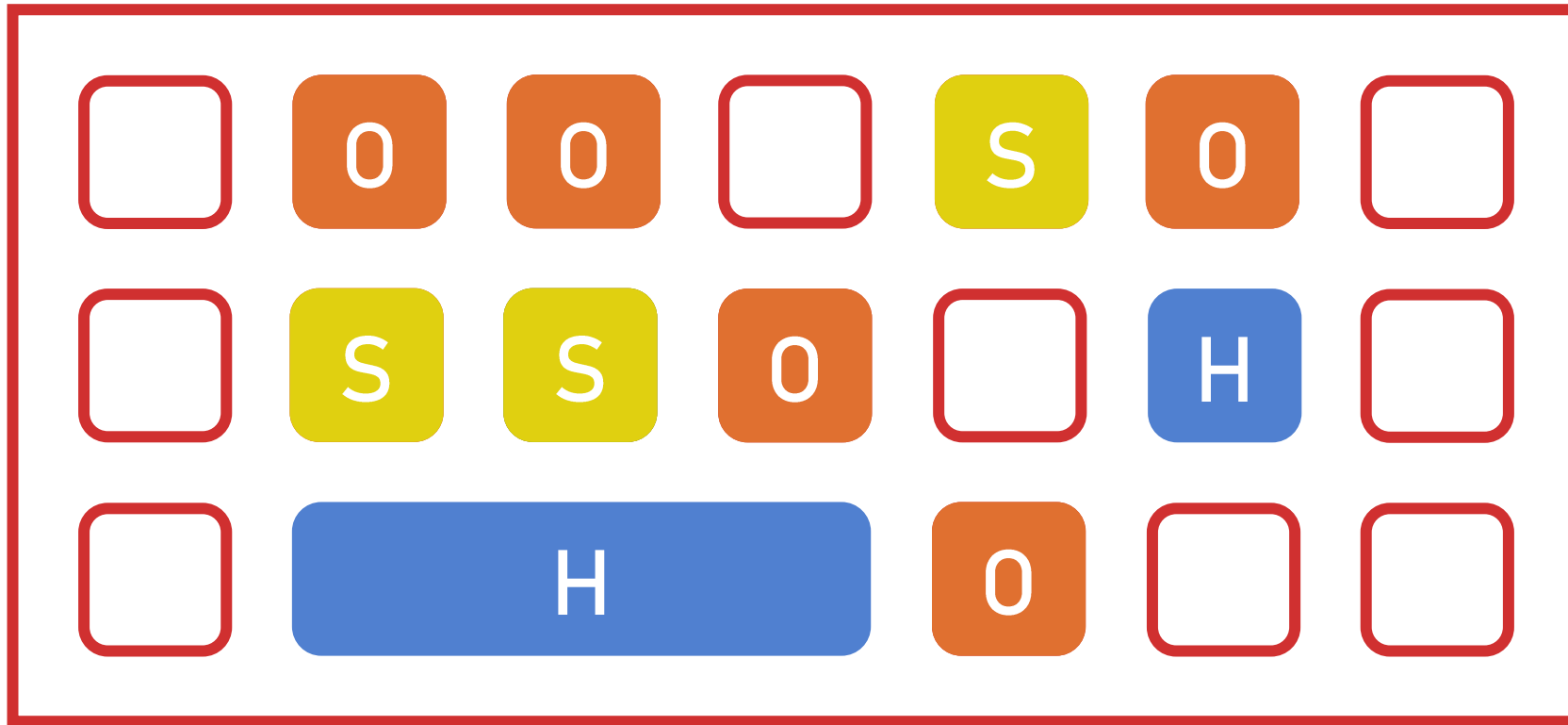


Old



Humongous

G1 GC – Mixed collection



Eden



Survivor

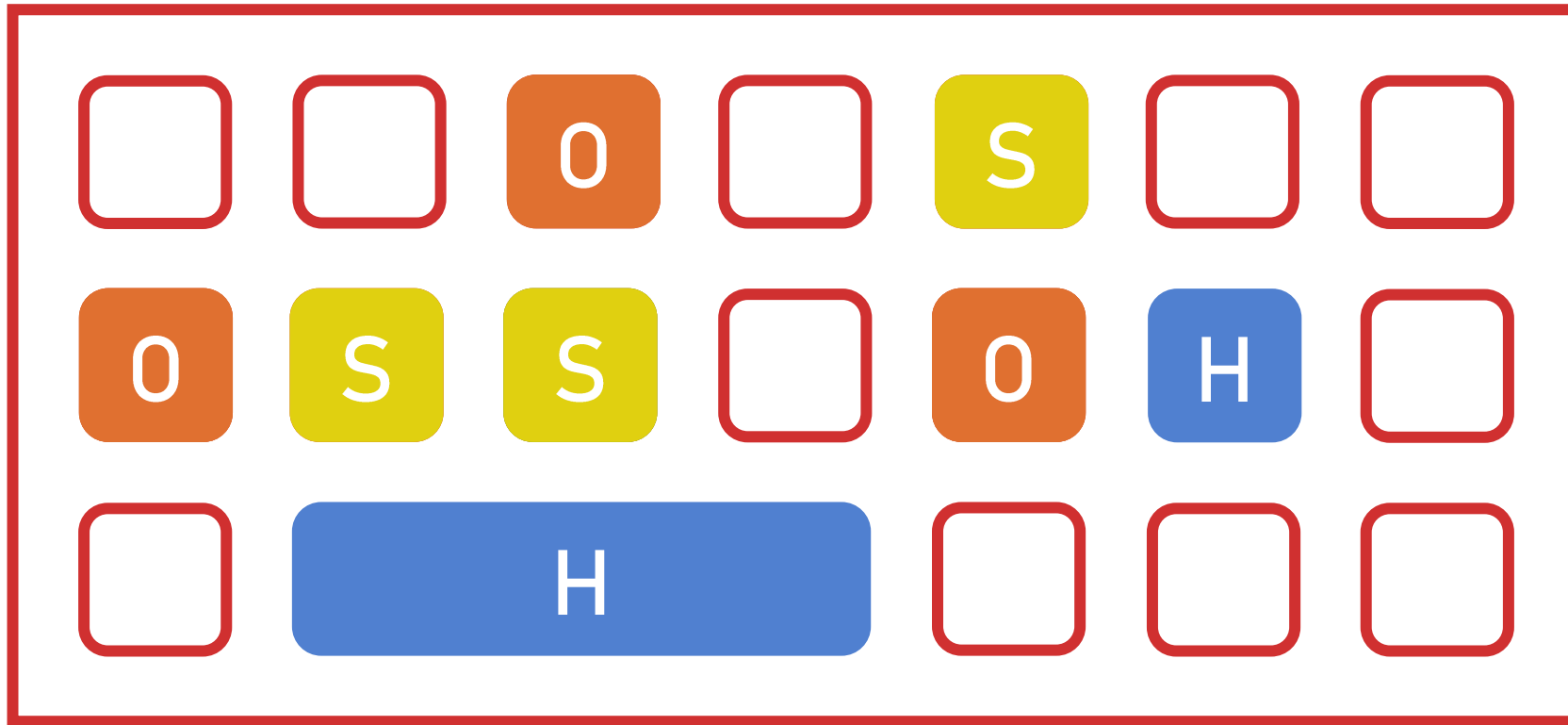


Old



Humongous

G1 GC – Mixed collection



Eden



Survivor

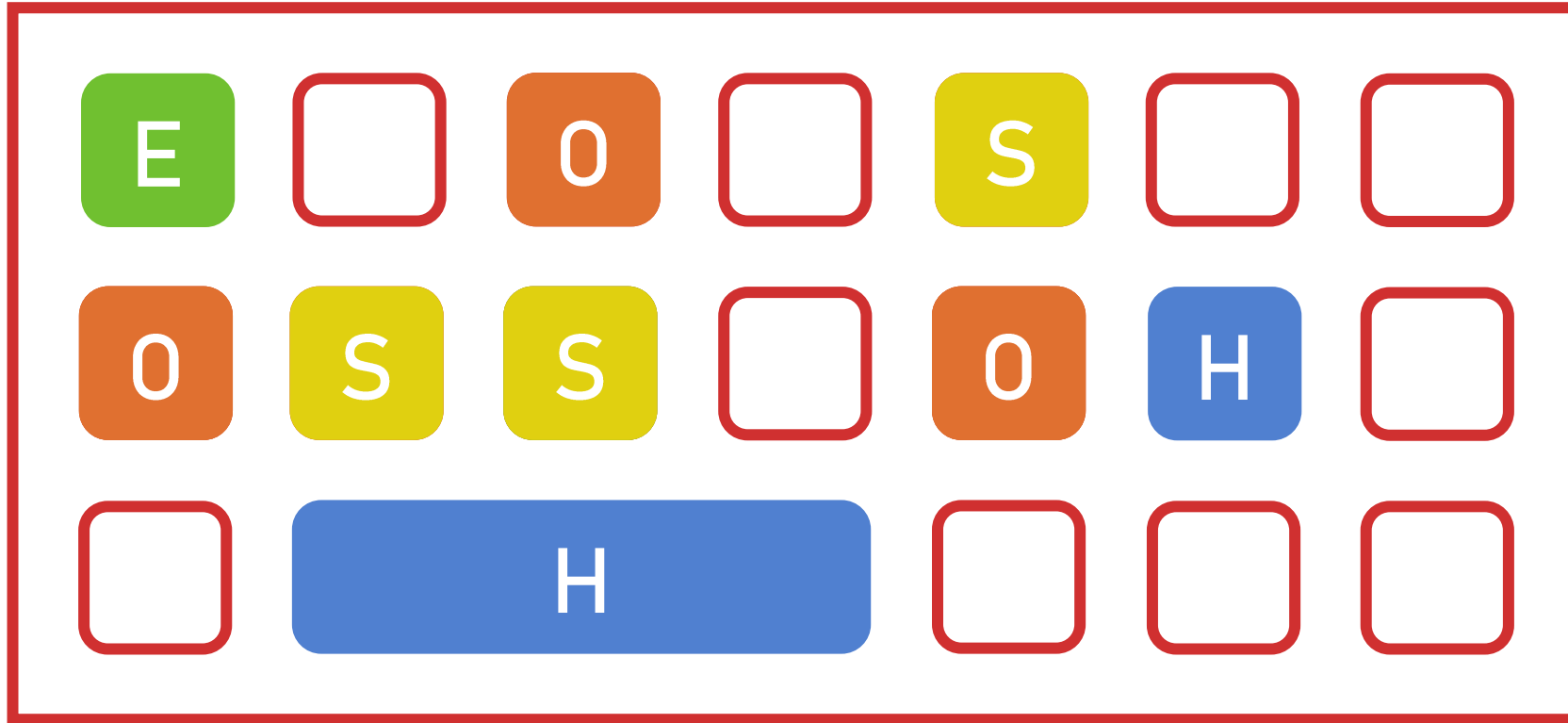


Old



Humongous

G1 GC



Eden



Survivor

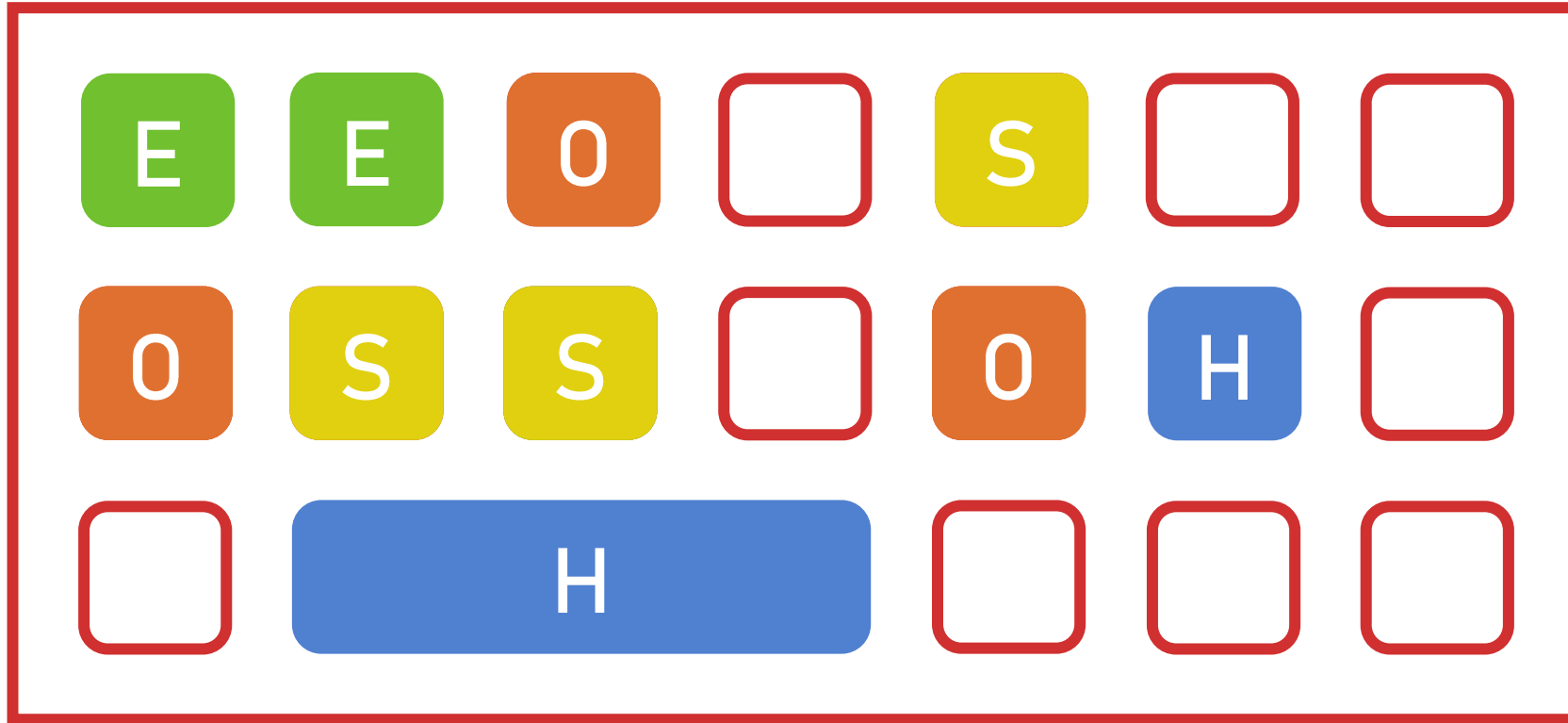


Old



Humongous

G1 GC



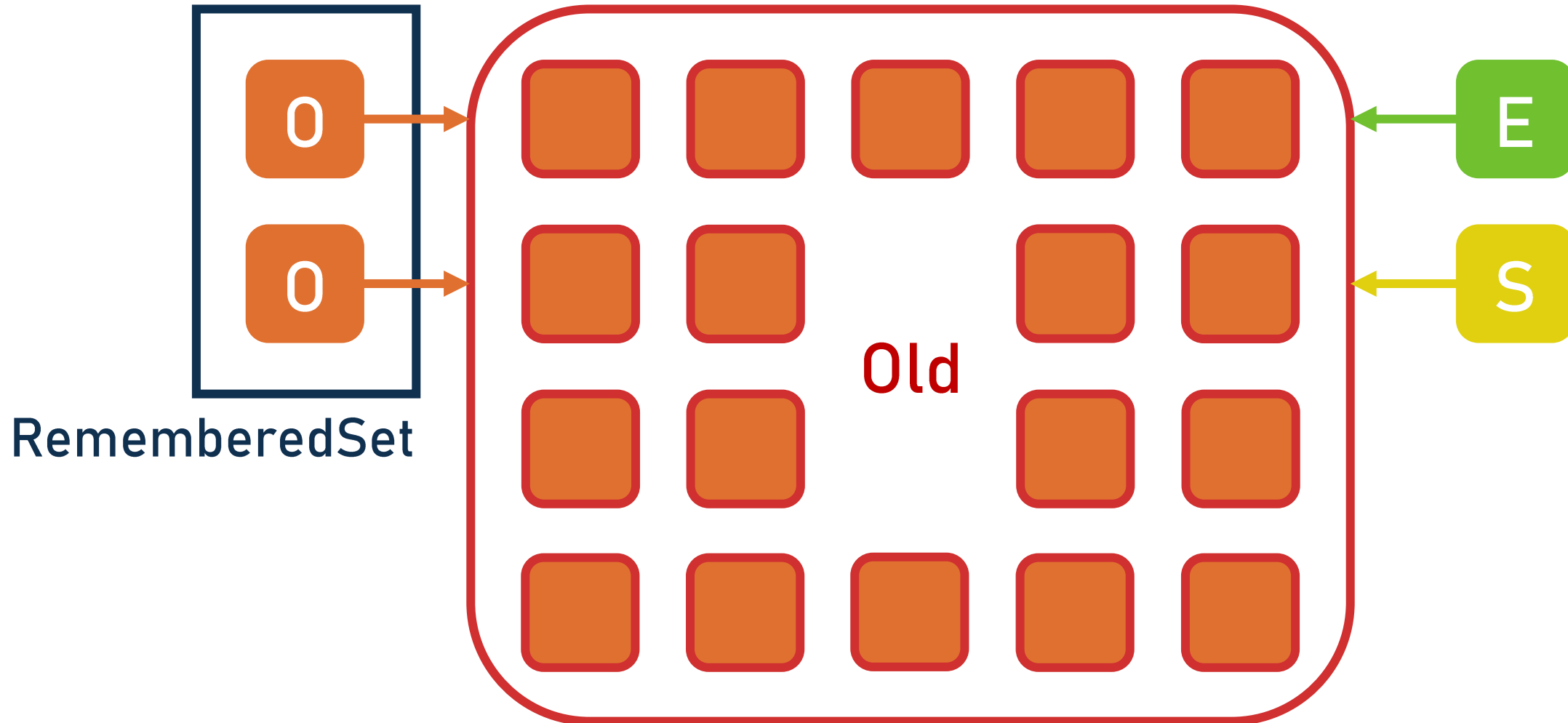
E Eden

S Survivor

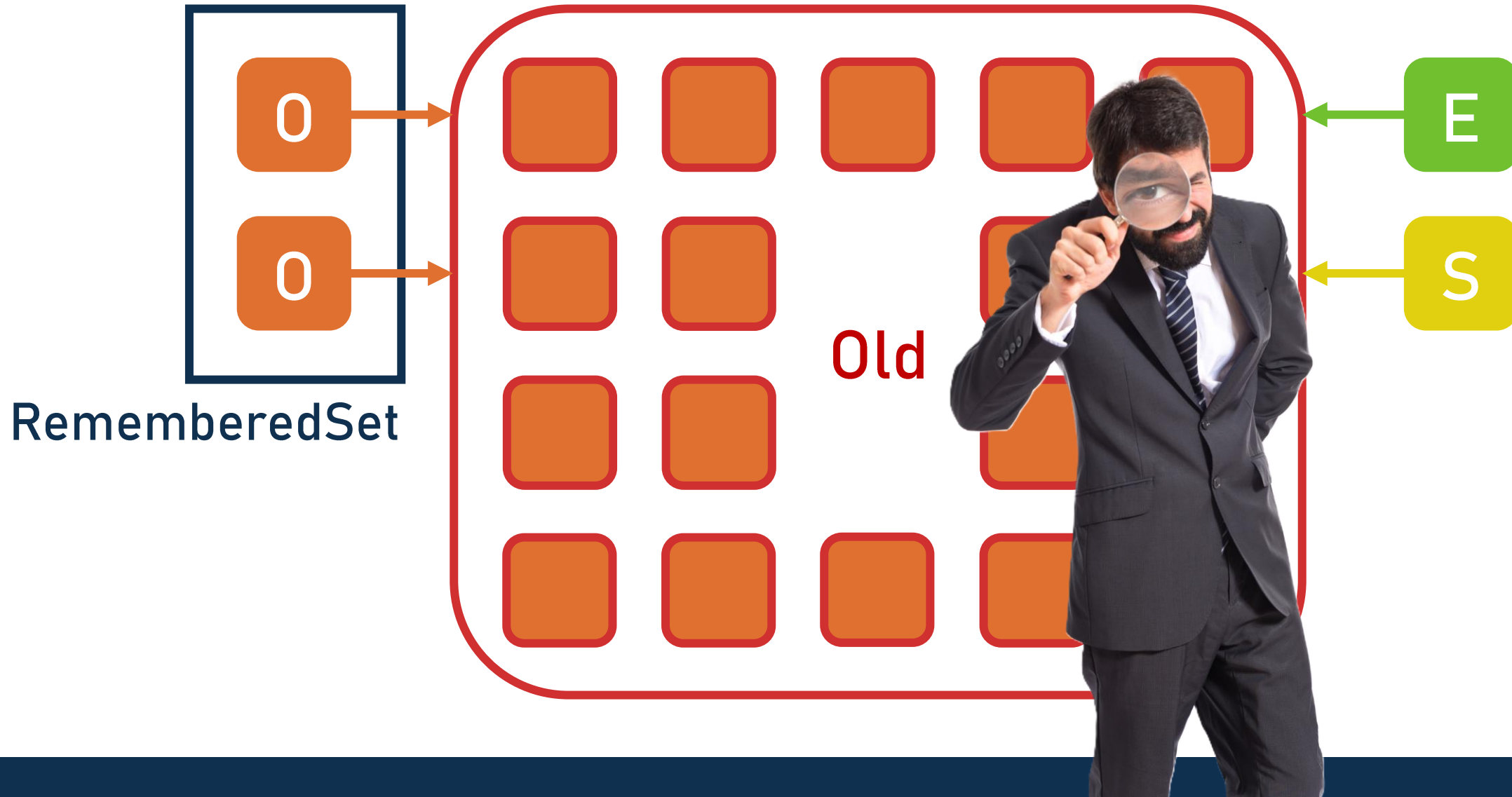
O Old

H Humongous

G1 GC – Region



G1 GC – Region



G1 GC



It starts with the regions holding the most garbage



It copies the live objects into other regions



Partially stop-the-world



It stops after 200 ms



It's the general-purpose GC

Latency?

Latency

Maximum pause: 200 ms **



* It can still do a full GC



** That's per JVM

you can hit a GC pause on every microservice

Ultra low latency ZGC and Shenandoah GC



Idea: store information inside the references

```
var conf = new Conference("Conference", 2026);
```



ZGC – Reference

	F	R	M1	M0	Address (44 bits)
--	---	---	----	----	-------------------

Address: up to 16 TB

4 colors

Memory (RSS) can be counted 3 times

Generational ZGC – Reference

	Address (46 bits)	12 colors
--	-------------------	-----------

12 colors, but still 2 instructions for load barrier

Address: up to 64 TB

Memory count is correct

ZGC



Auto-tuning



The application thread can do the update itself



64-bit references



Allocation stalls



Generational since Java 21

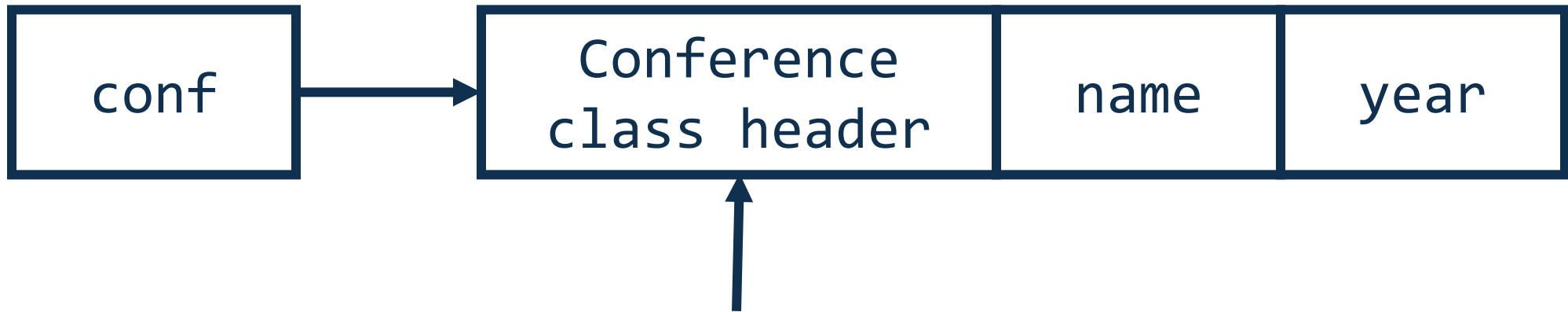
(non-generational removed in Java 24)

Shenandoah GC



Idea: store the information in the headers

```
var conf = new Conference("Conference", 2026);
```



Shenandoah GC – Header

	Identity hash		GC age		T	T	Class
--	---------------	--	--------	--	---	---	-------

Tags : 01/00/10 → locked/unlocked, 11 → GC

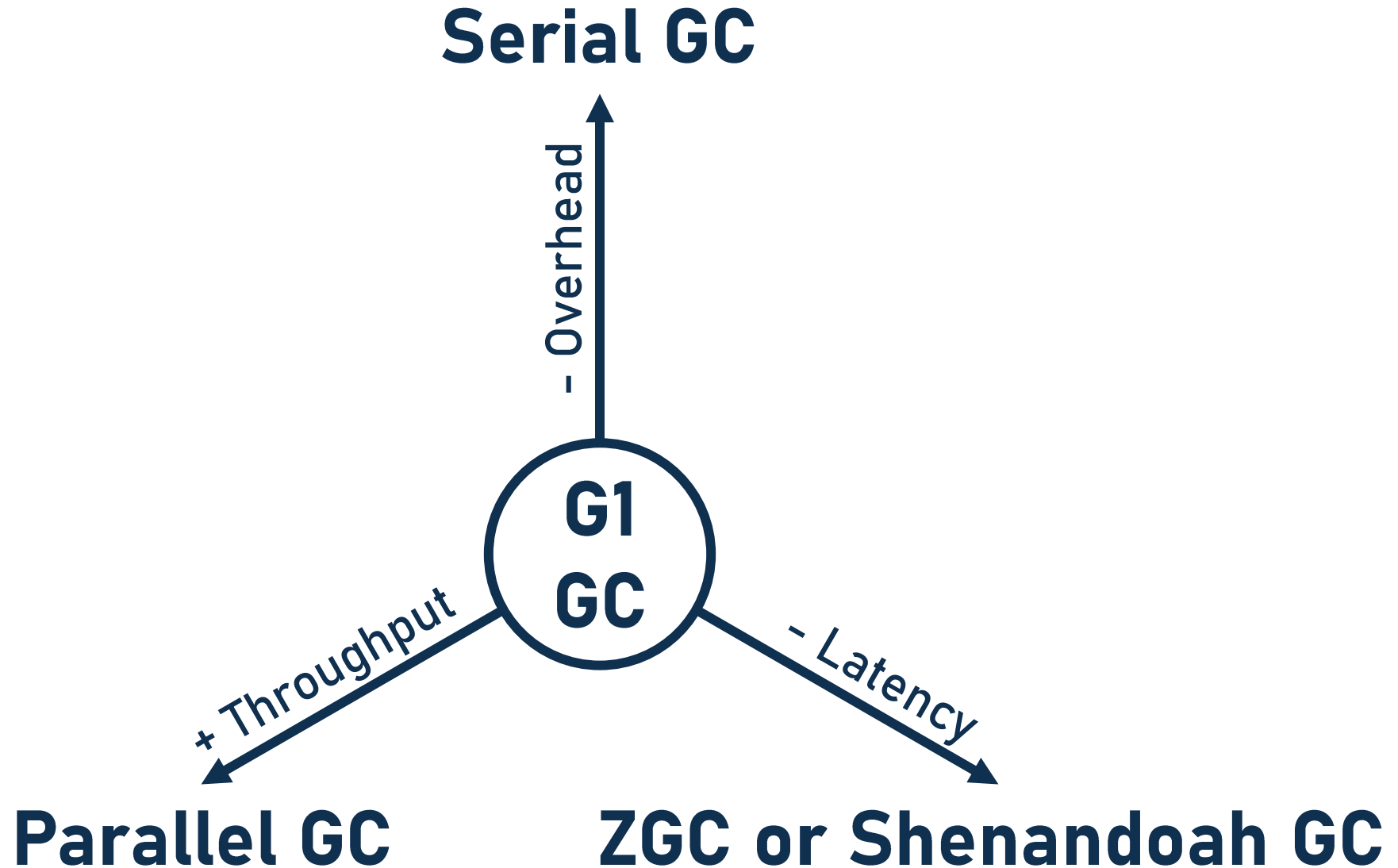
New address	1	1	Class
-------------	---	---	-------

Shenandoah GC

-  Highly configurable
-  No forwarding table
-  All references must be updated before reuse
-  Allocation stalls
-  Full GC
-  Generational since Java 25

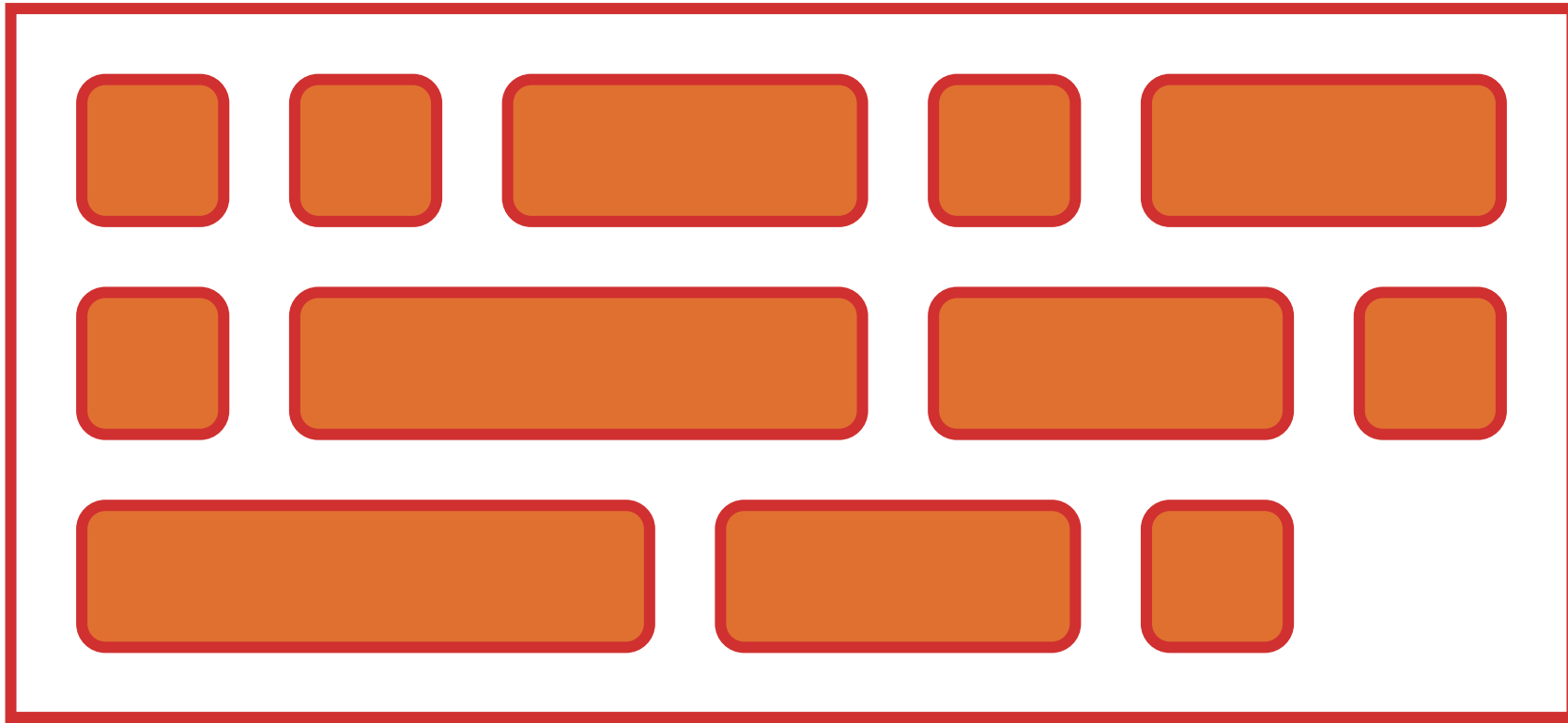
In summary

Which GC should you choose?

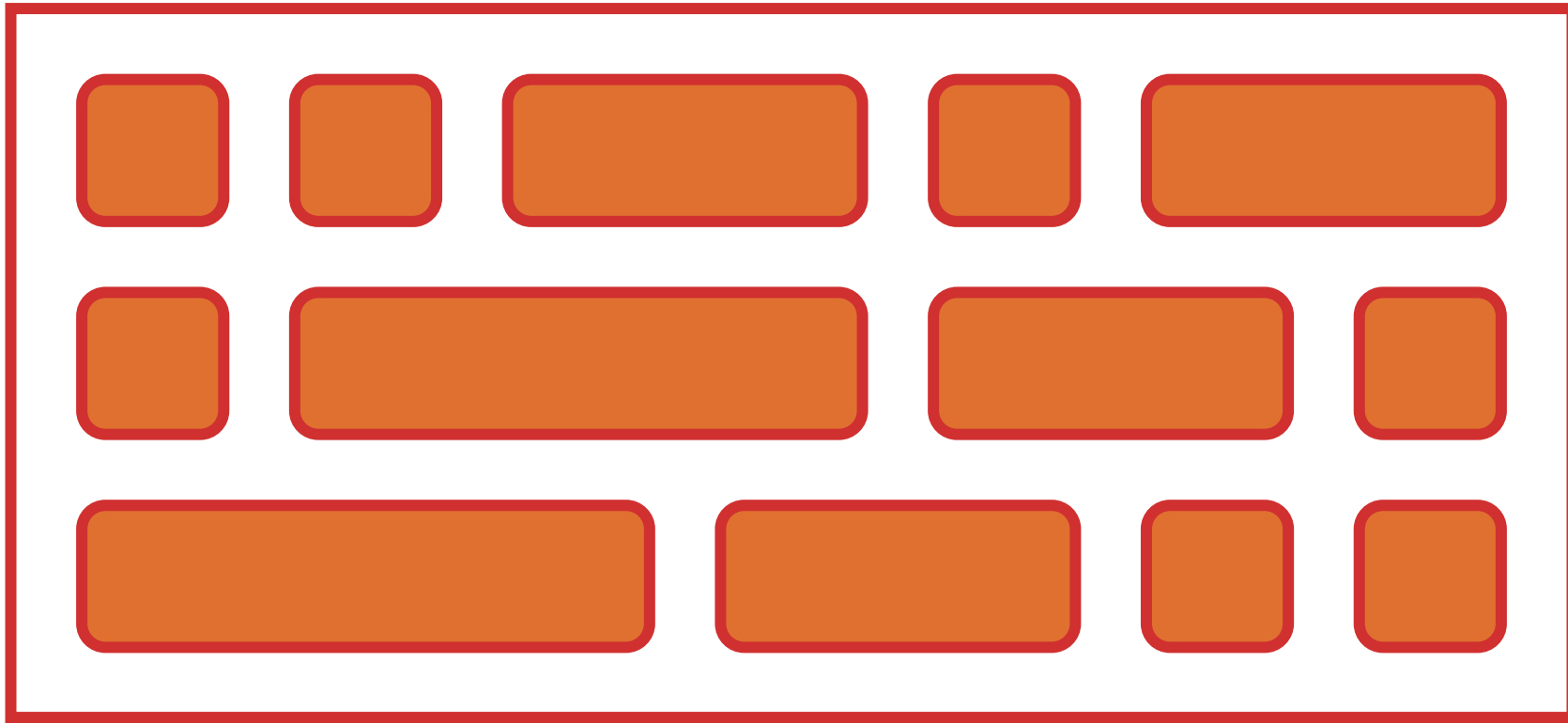


Epsilon GC ?

Epsilon GC



Epsilon GC





In summary

Which GC should you choose?

 Microservice (< 2 GB) → Serial GC

Which GC should you choose?

- 🐜 Microservice (< 2 GB) → Serial GC
- 💻 Serverless, CLI → Epsilon GC
- 📄 Batchs jobs, Kafka streams → Parallel GC
- 🕒 Latency is critical → ZGC or Shenandoah GC
- 👉 Every other case → G1 GC

In native Java

Native Java

GraalVM CE

Serial GC

Epsilon GC

Oracle GraalVM

Serial GC

Epsilon GC

G1 GC

The future

The future

- 💪 G1 throughput improvements (JEP-522, Java 26)
- 👉 G1 GC is always the default one (JEP-523, Java 27)
- 💡 Faster ZGC bootstrap (JEP-545)
- 💡 Automatic heap sizing (JEP-546 + drafts)
- 🌟 Thread-local ZGC

Performance

To improve performance

- 👉 Pick the GC that fits your need
- 👉 Measure
- 👉 Run a recent JVM

Magic flags

👉 `-Xms = -Xmx`

Magic flags

- 👉 `-Xms = -Xmx`
- 👉 `-XX:+UseLargePages`
- 👉 `-XX:+AlwaysPreTouch`
- 👉 Nothing else

Thank you

Slides



Antoine DESSAIGNE



Thanks